Secure and Role-Aware Explainable Occupant Tracking in Multi-Building Indoor Smart Spaces

Project report submitted in partial fulfilment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

in

COMPUTER SCIENCE AND ENGINEERING

Submitted by

Ajith Ashok AM.U4.U4CSE23204

Arjun Rajesh AM.U4.U4CSE23208

D Aditya Kiran AM.U4.U4CSE23215

Yadhu Vipin Madhilakath Madathil AM.U4.U4CSE23270

Under the guidance of

Dr. Lakshmi Mohan

Assistant Professor



AMRITA SCHOOL OF COMPUTING

AMRITA VISHWA VIDYAPEETHAM

AMRITAPURI CAMPUS

KOLLAM 690525

NOVEMBER 2026

AMRITA VISHWA VIDYAPEETHAM
DEEMED TO BE UNIVERSITY UNDER SECTION 3 OF UGC ACT, 1956

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Amrita School of Computing, Amritapuri Campus

BONAFIDE CERTIFICATE

This is to certify that the project report entitled "Secure and Role-Aware Explainable Occupant Tracking in Multi-Building Indoor Smart Spaces", submitted by Ajith Ashok (AM.U4.U4CSE23204), Arjun Rajesh (AM.U4.U4CSE23208), D Aditya Kiran (AM.U4.U4CSE23215) and Yadhu Vipin Madhilakath Madathil (AM.U4.U4CSE23270), in partial fulfilment of the requirements for the award of the degree of Bachelor of Technology in Computer Science and Engineering at Amrita Vishwa Vidyapeetham, is a bonafide record of the work carried out by them under my guidance and supervision at Amrita School of Computing, Amritapuri, during Semester 7 of the academic year 2026 - 2027.

Dr. Lakshmi Mohan
Project Guide
Signature _____

Project Coordinator
Signature _____

Chairperson, CSE
Signature _____

Reviewers
Signature _____

Place: Amritapuri
Date: 4 November 2026

AMRITA VISHWA VIDYAPEETHAM
DEEMED TO BE UNIVERSITY UNDER SECTION 3 OF UGC ACT, 1956

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Amrita School of Computing, Amritapuri Campus

DECLARATION

We, Ajith Ashok (AM.U4.U4CSE23204), Arjun Rajesh (AM.U4.U4CSE23208), D Aditya Kiran (AM.U4.U4CSE23215) and Yadhu Vipin Madhilakath Madathil (AM.U4.U4CSE23270), hereby declare that the project entitled "Secure and Role-Aware Explainable Occupant Tracking in Multi-Building Indoor Smart Spaces" is a record of our original work carried out under the guidance of Dr. Lakshmi Mohan, Assistant Professor, Department of Computer Science and Engineering, Amrita Vishwa Vidyapeetham, Amritapuri. To the best of our knowledge, this work has not formed the basis for the award of any degree, diploma, fellowship or similar qualification to any candidate at any university. All sources and contributions of others have been appropriately acknowledged.

Place: Amritapuri
Date: 4 November 2026

Student details	**Signature of the student**
Ajith Ashok AM.U4.U4CSE23204	_____
Arjun Rajesh AM.U4.U4CSE23208	_____
D Aditya Kiran AM.U4.U4CSE23215	_____
Yadhu Vipin Madhilakath Madathil AM.U4.U4CSE23270	_____

Project Guide: Dr. Lakshmi Mohan

Signature of the Project Guide: _____

ACKNOWLEDGEMENT

We thank our guide, Dr. Lakshmi Mohan, for proposing the problem, for the Distributed State Transition System framework on which this work builds, and for guiding every review from the first literature survey to the final evaluation. Her comments at the mid-review shaped the role-based access control, the explainability and the security work reported here.

We are grateful to the Department of Computer Science and Engineering and the Amrita School of Computing, Amritapuri Campus, for the computing facilities and the review process that kept the project on schedule. We also thank the authors of the open datasets and models we used, CASIA-WebFace, the Real-World Masked Face Dataset (RMFRD) and the InsightFace models, whose public release made this evaluation possible.

Finally, we thank our team members for the shared effort across the parts of the project: the routing and pipeline, the query, access-control and security layers, the explanation layer and the dashboard.

ABSTRACT

Multi-building campuses need to recognise and follow people across buildings, but keeping every face template in one central store concentrates sensitive biometric data, while keeping the templates apart makes it hard to find the building that knows a visitor. This project extends the Distributed State Transition System (DSTS) framework to decentralised multi-building occupant identification and tracking, with secure, role-based and explainable access to the results. Each building keeps its own gallery and publishes only a Bloom filter built from locality-sensitive-hash (LSH) codes of its occupants' face embeddings. A capture that is not recognised locally is routed to a short list of likely buildings, which verify it by angular-distance voting. A dynamic visitor database, per-state probabilistic tracking, two-layer role-based access control with a class-teacher model, sealed inter-building transfers and explanations redacted to the caller's disclosure level complete the system. It is evaluated on 550 simulated occupants in 11 buildings (500 from CASIA-WebFace and 50 masked people from the real-world RMFRD dataset) over 11,000 events per simulated day, against a centralised search, and averaged over five random populations. The decentralised pipeline identifies 91.5% of events against 92.2% for the central search while touching 10.3% of the galleries; building-level state F1 is 0.950. Unmasked people are identified 94.8% of the time and masked people 59.1%. The uncertainty flags attached to answers catch 74% of wrong identifications at a cost of about 8% extra latency, and 571 automated tests cover the policy, secure-transfer and explanation layers. The network, cameras and certificate authority are simulated, so the work demonstrates the design and its accuracy rather than a hardened deployment.

Keywords: Occupant identification and tracking; Distributed state transition system; Locality-sensitive hashing; Bloom filter; Role-based access control; Explainable queries; Masked face recognition

Table of Contents

Acknowledgement **i**

Abstract **ii**

List of Figures **v**

List of Tables **vi**

1 Introduction **1**

1.1 Background and motivation 1

1.2 Objectives 2

1.3 Scope and constraints 2

1.4 Project report organization 4

2 Problem Definition **5**

2.1 Problem statement 5

2.2 Existing system and limitations 6

3 Literature Review **7**

3.1 Review scope and selection 7

3.2 Descriptive review of literature works 7

3.3 Comparison of reviewed literature 14

4 Requirements **17**

4.1 Functional requirements 17

4.2 Non-functional requirements 19

4.3 Hardware requirements 19

4.4 Software and data requirements 19

5 Proposed Methodology **22**

5.1 Design (architecture) or conceptual framework 22

5.2 Proposed approach and procedure 25

6 Results and Analysis **33**

6.1 Evaluation setup 33

6.2 Evaluation criteria 34

6.3 Result 34

6.4 Comparative Analysis 45

6.5 Discussion and Interpretation 48

6.6 Limitations 48

7 Conclusion **50**

7.1 Conclusion 50

7.2 Future work 50

References **52**

A Source Code and Reproducibility **55**

A.1 Setup and execution 55

A.2 Supporting material 56

List of Figures

5.1 System architecture: building nodes, secure communication layer, access control layer and user interface 22

5.2 Inter-building tracking workflow and querying and access workflow 23

6.1 Identification accuracy, centralised search against the decentralised pipeline (average of five seeds) 35

6.2 Outcome of the 2,200 visitor events in the detailed run (seed 42) 37

6.3 Precision, recall and F1 of the tracked state at three levels (average of five seeds) 38

6.4 Share of events identified correctly, masked against unmasked people (average of five seeds) 39

6.5 F1 of masked probes for the tested variants (offline, building 11) 40

6.6 Left: share of wrong and of correct identifications carrying a flag. Right: extra latency of building the explanation, per query type (average of five seeds) 43

6.7 Configured and measured false-positive rates of the building filters 43

6.8 Campus overview: 11 buildings and 550 occupants, with building 11 marked as the all-masked building 46

6.9 A Dean's query with its "Why this answer" card (rule, evidence, uncertainty, withheld) 46

6.10 The same page for a Teacher: the answer is reduced and the card states what was withheld 47

List of Tables

1.1 Objectives, evaluation methods and deliverables 3

2.1 Stakeholders and what they need from the system 5
2.2 Design alternatives for finding a visitor's home building 6

3.1 Comparison of reviewed literature 14

4.1 Functional requirements 17
4.2 Use cases 18
4.3 Non-functional requirements 20
4.4 Software, tools and data 20

5.1 Components and their responsibilities 24
5.2 What a building keeps and what it shares 24
5.3 Mask handler parameters (defaults) 27
5.4 The three layers of the visitor database 28
5.5 Disclosure levels 28
5.6 Access model (class-teacher RBAC) 29
5.7 Query templates 29
5.8 Uncertainty flags 30
5.9 Security mechanisms 31
5.10 Parameters used 32
5.11 Main API endpoints 32

6.1 Evaluation protocol 33
6.2 Identification accuracy and gallery touched (average of five seeds, 11,000 events per run) 35

6.3 Event-level confusion counts and scores (average of five seeds; TN, in the millions, omitted) . . . 35
6.4 Visitor identification and tracking (average of five seeds) . . . 36
6.5 Results per random population (11,000 events each) . . . 37
6.6 State-level scores against the state ground truth (average of five seeds) . . . 37
6.7 Masked against unmasked people (average of five seeds) . . . 38
6.8 Masked probes against the 50-person gallery: fractions of the 1,000 probes (offline, building 11) . . . 39
6.9 Rank-1 accuracy of masked probes without thresholds (50-way) . . . 40
6.10 Uncertainty flags on identifications (average of five seeds) . . . 41
6.11 Individual flags on identifications (average of five seeds) . . . 41
6.12 Uncertainty flags on answers, per query type (average of five seeds) . . . 41
6.13 Cost of explanations per query (average of five seeds; 100 runs each) . . . 42
6.14 False-positive rates of the building filters (average of five seeds) . . . 43
6.15 Automated tests by area . . . 44
6.16 Representative decisions from the persona matrix . . . 45
6.17 Comparison with the baseline and the approaches in the literature . . . 47

7.1 Objective-wise status . . . 51

A.1 Main commands . . . 56
A.2 Repository layout . . . 57
A.3 Outputs of one run (runs/seed_XX/) . . . 57
A.4 Source of the reported results . . . 58
A.5 Visitor outcome buckets, seed 42 . . . 58

Chapter 1

Introduction

1.1 Background and motivation

Smart spaces use cameras, sensors and computing to observe who is where and to support occupant-specific services such as access, scheduling and safety [6, 7, 26]. On a university campus or a corporate estate these services span many buildings, so the system must keep identifying and following a person as they walk from one building to the next. Camera networks for indoor tracking are well studied [1, 2, 3, 4, 5], but the identity question changes when the space has several buildings: a person registered in one building is a *visitor* in every other, and the building that captures the face does not hold that person's reference photos.

There are two obvious designs. A *centralised* design keeps every person's reference embeddings in one store and searches all of them for every capture. It is simple and accurate, but it concentrates biometric data, which cannot be reissued if it leaks, and it makes one server the single point of failure and of attack. A *distributed* design keeps each building's reference data in that building. This respects the locality of the data, but a visitor's home building is unknown, so the capturing building must either contact every other building for every unrecognised face or share reference embeddings between buildings [6]. The base paper that this project extends, the Distributed State Transition System (DSTS) of Mohan and Menon [6], resolves visitors through a central master list.

A second gap concerns the people who ask questions of the tracking data. The same data that lets a Dean find a lost visitor also lets any user follow any other person, so the system must decide who may see whom, and at what precision [21], and must be able to show why an answer was given. Access control for tracking queries and explanations of the answers are rarely treated together with distributed identification.

This project therefore combines three things in one system: (i) decentralised identification and tracking in which only a one-way Bloom filter leaves a building until a match is confirmed; (ii) role-based, privacy-aware querying with a two-layer policy; and (iii) explanations

that justify both the access decision and the tracking evidence behind each answer. A first, narrower study of the routing step alone has been written up as a conference-style manuscript [28]; this report covers the complete system, including state tracking, access control, security, explanations and a masked-face building.

Relevant Sustainable Development Goals. The work maps to three goals of the 2030 Agenda [27]. *SDG 9 (Industry, Innovation and Infrastructure)*, target 9.1 on quality, reliable and resilient infrastructure: the system is a resilient, decentralised infrastructure for smart buildings with no central biometric store to lose. *SDG 11 (Sustainable Cities and Communities)*, target 11.7 on safe and inclusive spaces: occupants of a campus can be located for safety and service while the data is protected from misuse such as stalking. *SDG 16 (Peace, Justice and Strong Institutions)*, target 16.6 on effective, accountable and transparent institutions: every access decision is audited and every answer is explained. The intended beneficiaries are campus administrators, who get reliable tracking, and the occupants, whose biometric data stays in their own building and whose location is visible only to people with a legitimate relationship to them.

1.2 Objectives

The objectives, each linked to how it is evaluated and to its deliverable, are given in Table 1.1.

1.3 Scope and constraints

Intended users. Campus administrators (Deans), faculty (Teachers) and students; each sees tracking data only as their role and relationship allow. **Included functions.** Face-embedding based identification of registered occupants and of visitors across buildings; probabilistic zone-level tracking; thirteen query templates (Q1–Q13) and a full-trajectory query; role-based disclosure; explanations; secure inter-building exchange; a web dashboard; and a centralised baseline for comparison.

Exclusions and assumptions. The system works on face embeddings that are produced by a pretrained model; it does not train a face model and does not process live video. Cameras, the network between buildings and the certificate authority are simulated in one process. The occupant population is synthetic in its movements: 20 reference and 20 test photos per person come from public datasets, and the test photos are replayed as camera events. Zones are a fixed nine-zone layout per building. Every visitor is a person registered in some other building; people who are registered nowhere (unknown visitors) are outside the scope and are listed as future work. **Constraints.** All experiments run on a laptop CPU; every result is reproducible from a population seed.

Table 1.1: Objectives, evaluation methods and deliverables

#	**Objective**	**Evaluation method**	**Deliverable**
1	Extend the DSTS framework to distributed multi-building identification and tracking	Event identification precision and recall; state-level precision and recall over every state (Sections 6.3.1, 6.3.4)	Pipeline, building nodes, BSTS state store
2	Provide an RBAC query interface for Dean, Teacher and Student roles	Every policy cell tested; access matrix checked against the expected decisions (Section 6.3.9)	Two-layer policy, FastAPI server, dashboard
3	Return explainable responses that justify the access decision and the tracking evidence	Faithfulness and no-leakage tests; flag precision and recall against ground truth; latency overhead (Section 6.3.7)	Explanation model, redaction, "Why this answer" card
4	Design LSH + Bloom-filter routing for decentralised visitor lookup	Routing recall; gallery touched per lookup; accuracy against a centralised search; effective false-positive rate (Sections 6.3.1, 6.3.2, 6.3.8)	Enrolment, routing and verification modules
5	Secure the system: sealed transfers, role-based filtering, audit logging	Tamper, replay, wrong-sender and fail-closed tests; login lockout tests (Section 6.3.9)	secure_transfer and access_control packages
6	Evaluate tracking precision and recall, communication cost and explanation quality	TP/FP/FN/TN per cohort and level; five seeds; centralised baseline; explanation measurements (Chapter 6)	Evaluation scripts and result tables

1.4 Project report organization

Chapter 2 states the problem and the limits of existing systems. Chapter 3 reviews fourteen related works and compares them. Chapter 4 lists the functional, non-functional, hardware and software requirements. Chapter 5 presents the design and the step-by-step approach, including the algorithms, the mask-handling logic and the parameters. Chapter 6 describes the evaluation, reports the results with tables and figures, covers the explanation measurements and the Bloom-filter finding, compares the system with a centralised baseline and with the earlier routing study, and discusses limitations. Chapter 7 concludes and lists future work. The references and an appendix on source code and reproducibility follow.

Chapter 2

Problem Definition

2.1 Problem statement

Problem. Identify and track occupants across a wide-area, multi-building indoor environment while giving secure, role-based and explainable access to where they are and have been, and justify both the access decisions and the tracking evidence behind each answer.

Stakeholders and consequences. Table 2.1 lists who is affected and what goes wrong if the problem is left unsolved. If identification is centralised, one breach exposes everyone's biometric data. If it is distributed without routing, every unrecognised face costs a request to every building. If access is not controlled, tracking becomes a stalking tool. If answers are not explained, they cannot be trusted or audited.

Table 2.1: Stakeholders and what they need from the system

Stakeholder	**Need**	**Risk if unmet**
Occupants	Biometric data stays in their own building; location visible only to people with a legitimate relationship	Breach of a central store; being followed by a peer
Deans (administrators)	Find a visitor or a lost person; follow own campus; audit access	Slow, costly or unreliable searches
Teachers	Follow only the students on their class roster, in any building	Over-sharing of student movements
Students	See where their assigned teachers are; see themselves in full	Peer surveillance
Auditors	Know who asked what, and why an answer was given	Answers that cannot be justified

Inputs and outputs. The identification input is a face embedding captured at a building, zone and time; the output is an identity (or an abstention) and an updated probability of the person being in each of the nine zones. The query input is a request from a logged-in user (role, target, time or interval, purpose); the output is the permitted answer together with its

explanation, or a refusal with a reason code.

2.2 Existing system and limitations

Existing indoor tracking systems use cameras, radio signals or both [1, 2, 9]. Edge-camera frameworks localise and track people accurately inside one site [1, 2, 5], and distributed trackers associate targets across cameras with consensus filtering or shared target tables [3, 4]. These systems stay within one site. They do not address how the identity of a person who is registered elsewhere is discovered, and they do not define who may query the resulting locations.

The DSTS framework [6] models a multi-building environment as states, events and transition functions and resolves visitors through a centralised master list. Bloom filters have been used to protect biometric templates [10], and a recent privacy-preserving crowd-counting system combines face features, fuzzy extractors and encrypted Bloom filters [11]. Summary Cache [13] uses Bloom filters to find the web proxy likely to hold an object. None of them combines approximate, similarity-based routing of a face to a building with role-based access control and explanations. Table 2.2 contrasts the design alternatives.

Table 2.2: Design alternatives for finding a visitor's home building

Design	**How a visitor is found**	**Biometric data**	**Cost per unknown face**
Centralised store	One global search over all galleries	All galleries pooled in one place	One request, one large search
Distributed, no routing	Ask every other building	Stays local, but the face goes to every building	$B-1$ requests (10 here)
Master list [6]	Look the visitor up in a central list of visitor embeddings	Visitor embeddings pooled centrally	One request to the list
LSH + Bloom routing (this work)	Rank buildings by Bloom score, verify the top five	Only a one-way Bloom filter is shared; references move after a confirmed match	About two requests per routed lookup

Gap. Four gaps remain: (i) identity discovery across separate buildings without sharing galleries; (ii) evidence that LSH with Bloom filters can route biometric visitors accurately; (iii) role-based control over tracking queries; and (iv) explanations for access decisions and tracking answers. This project addresses all four.

Chapter 3

Literature Review

3.1 Review scope and selection

Fourteen works (references 1–6, 8 and 10–16) were reviewed in twelve groups (Section 3.2); they are cited by their reference number throughout the report. The search covered indoor occupant tracking with camera and RF sensing, multi-camera association, state-transition models of smart spaces, privacy-preserving biometrics and Bloom-filter indexing, and similarity search with locality-sensitive hashing. Search keywords were: "indoor localization camera network edge", "distributed multi-camera multi-target tracking", "smart space state transition", "Bloom filter biometric", "locality sensitive hashing" and "masked face recognition dataset".

The publication period is 1970 to 2026: recent works (2020–2026) on tracking and privacy-preserving biometrics were chosen for the problem context, and foundational works on Bloom filters and hashing were included because the routing mechanism is built on them. A work was selected if it is a primary source (a journal article, conference paper or preprint), is directly relevant to tracking, identity discovery, hashing-based routing, or the models used in this project, and gives enough detail to describe its approach and evaluation. Works [1]–[5] are the mid-review set; [6] is the base paper that this project extends. The remaining references (the datasets, face models, access-control model and cryptographic standards used in Chapters 4 and 5) are cited where they are used and are not reviewed here.

3.2 Descriptive review of literature works

Each group below gives one connected paragraph covering the problem or objective, the proposed contribution and approach, the study context, the evaluation criteria, the key findings and the limitations that leave a gap for this project. Table 3.1 compares all of them.

3.2.1 Group identification and localization with privacy-preserving edge cameras

Hegde et al. (2024) in "Indoor group identification and localization using privacy-preserving edge computing distributed camera network" [1] investigate how to find when and where groups form in an indoor therapeutic space of about 1,700 m^2. The work proposes a pipeline that takes noisy positions and orientations of people, estimated from sparsely placed multi-view edge cameras running a lightweight two-dimensional pose detector, and groups them. The study is situated in a therapeutic built environment with seven participants over 35-minute sessions in groups of two to seven. The authors assess it with precision, recall and F1 for group identification and mean absolute error for group localization. They report F1 of up to about 90% and a localization error of 1.25 m, with raw images kept off the network because the edge devices extract the features. The limitations are that the unit of identification is a group and not an individual, and that the study covers one space; it says nothing about identifying a person who is registered elsewhere.

3.2.2 Indoor localization and multi-person tracking with sparse edge cameras

Kwon et al. (2023) in "A feasibility study on indoor localization and multiperson tracking using sparsely distributed camera network with edge computing" [2] ask whether people can be localized and tracked over a large indoor site with few, low-cost cameras. They deploy 39 edge camera units, each running pose estimation on a Google Coral accelerator, in a therapeutic space of roughly 1,700 m^2, and track people from the pose output. The evaluation reports localization error, multiple-object tracking accuracy, tracking precision, identity switches and body-orientation error, with an average localization error of about 1.4 m and a tracking accuracy of about 88.6%. The framework tracks anonymous people inside one site and does not attempt cross-building identity discovery, access control or explanations.

3.2.3 Distributed multi-camera multi-target association

Yang et al. (2022) in "Distributed multi-camera multi-target association for real-time tracking" [3] address real-time association of targets across cameras without a central server. They propose a target-management module in which cameras share a data structure of target labels and appearance information so that detections in different views can be matched, without camera calibration. The evaluation uses several multi-camera benchmarks and an own mobile-camera dataset and reports multi-camera tracking accuracy, IDF1, false positives, false negatives and identity switches. The work improves association, but it keeps no probabilistic state of who is in which building and does not identify where a visitor comes from.

3.2.4 Distributed multi-target tracking in camera networks

Casao et al. (2021) in "Distributed multi-target tracking in camera networks" [4] study tracking of several people with a network of cameras that exchange information only with neighbours. Their approach combines a distributed-consensus Kalman filter with a re-identification network and a distributed tracker manager that handles cross-camera association. They compare against centralised trackers on public multi-camera datasets using tracking accuracy, IDF1-type scores and bandwidth, and report better accuracy and lower bandwidth than the centralised methods they compare with. The gap is that the system tracks within one camera network and has no model of buildings, registered occupants and visitors.

3.2.5 Real-time multi-person multi-camera tracking at the edge

Gaikwad and Karmakar (2021) in "Smart surveillance system for real-time multi-person multi-camera tracking at the edge" [5] build an end-to-end pipeline of detection, tracking and re-identification that runs on an NVIDIA Jetson TX2 edge device. It is evaluated on the authors' own multi-camera campus recordings, collected over a full day, and reports an IDF1 score of 90.97, about 30 frames per second and a re-identification latency of roughly 90 ms. The pipeline shows that edge tracking and re-identification are practical, but it has no formal multi-building state or query model and no decentralised handling of visitors.

3.2.6 Modelling large-scale camera networks: the DSTS framework

Mohan and Menon (2020) in "Modelling large scale camera networks for identification and tracking: an abstract framework" [6] model identification and tracking in a wide-area camera network as a distributed state transition system. A state records the probability that each occupant is in each zone, an event is a recognition at a zone, and a transition function updates the state when an event occurs; each building has a building-level state transition system (BSTS). Registered occupants are recognised against their own building's database, and visitors are resolved through a centralised master list. The framework is abstract and the visitor step is the part this project replaces: the central list concentrates biometric data and every visitor lookup depends on it. The model of states, events and transitions builds on earlier work on the three Rs of cyber-physical spaces [7].

3.2.7 Face recognition and tracking for security surveillance

Nair et al. (2021) in "Face recognition and tracking for security surveillance" [8] investigate identifying and following faces across surveillance cameras, including cameras with non-overlapping views. The work combines face detection and recognition with tracking so that a person recognised in one camera can be followed into the next. It shows that face recognition is a practical identity cue for tracking, which is the assumption this project relies on. As a surveillance system it is not designed around separate per-building galleries, cross-building identity discovery or role-based querying.

3.2.8 Bloom filters for iris biometrics

Rathgeb et al. (2014) in "On application of Bloom filters to iris biometrics" [10] investigate how Bloom filters can hold iris templates. Representing the template as a Bloom filter makes comparison alignment-free and one-way, so that the stored template protects the biometric while still allowing identification, and the structure supports faster searches than a linear scan of templates. The work establishes that Bloom filters are a usable representation for biometric data. It targets iris codes, not face embeddings, and it protects a single database rather than routing a query among many databases.

3.2.9 Privacy-preserving face-based crowd monitoring

Marzani et al. (2026) in "Head count: privacy-preserving face-based crowd monitoring" [11] address counting individuals across locations without revealing their faces. They combine face features, fuzzy extractors and homomorphically encrypted Bloom filters so that the same person seen in different places is counted once while the underlying biometric stays hidden. This is the closest related work to using Bloom filters for cross-location face matching. Its purpose is counting, not identification and tracking of named occupants, and it has no role-based query layer; this project uses LSH codes and per-building Bloom filters to route a capture towards the building that holds the reference.

3.2.10 Space/time trade-offs in hash coding

Bloom (1970) in "Space/time trade-offs in hash coding with allowable errors" [12] introduces the Bloom filter: a bit array and several hash functions that answer "probably in the set" or "definitely not in the set", with a controllable false-positive rate and no false negatives. The analysis trades a small error probability for a large saving in space. This is the property the project depends on: each building can publish a compact, one-way summary of its occupants. The filter answers exact membership only, which is why locality-sensitive hashing is needed to make nearby embeddings collide; and, as Section 6.3.8 shows, the analysis assumes independent items, which LSH codes are not.

3.2.11 Summary Cache: Bloom filters for wide-area cache sharing

Fan et al. (2000) in “Summary Cache: a scalable wide-area web cache sharing protocol” [13] study how web proxies can share cache contents without flooding each other with queries. Each proxy keeps a Bloom-filter summary of the objects held by its peers and contacts only the peers whose summaries match, which sharply reduces inter-proxy messages at a small cost in false hits. The evaluation is on trace-driven simulations of proxy traffic. The same idea of summaries that tell where an item probably lives is applied here to faces, but a face capture never matches exactly, so the summaries must be built from similarity-preserving codes.

3.2.12 Similarity-preserving hashing: SimHash and multi-probe LSH

Charikar (2002) in “Similarity estimation techniques from rounding algorithms” [14] shows that random-hyperplane hashing maps vectors to bits so that the probability of two vectors agreeing on a bit grows with the angle between them, which makes binary codes preserve angular similarity. Lv et al. (2007) in “Multi-probe LSH: efficient indexing for high-dimensional similarity search” [15] improve recall without more hash tables by also probing the buckets that differ from the query in the least confident bits. Kirsch and Mitzenmacher (2006) show that Bloom-filter positions can be derived by double hashing with no loss in asymptotic false-positive rate [16]. These three works supply the encoding, multi-probe lookup and hashing used in this project. They are generic indexing techniques and say nothing about biometric privacy or access control.

3.3 Comparison of reviewed literature

Table 3.1 compares the reviewed works on the attributes used above. The gap that follows from the comparison is stated here, and the table gives the evidence. **Gap.** None of the surveyed works routes a face from a building that does not know it to the building that does, using compact similarity-preserving summaries, and none combines this with role-based access to the tracking data and with explanations of the answers. That combination is the contribution of this project.

Table 3.1: Comparison of reviewed literature

Work and citation	**Problem or objective**	**Approach and contribution**	**Study context**	**Evaluation criteria**	**Key findings**	**Limitations and gap**
Work 01 [1] Hegde et al., 2024	Find when and where groups form indoors	Pose estimation and clustering on edge cameras	One therapeutic space, 7 people, 35 min	Precision, recall, F1, MAE	F1 up to about 90%, MAE 1.25 m	Groups, not individuals; one site
Work 02 [2] Kwon et al., 2023	Localize and track people with few cameras	39 edge units with pose estimation	About 1,700 m^2 therapeutic space	Localization error, MOTA, MOTP, ID switches	About 1.4 m error, MOTA about 88.6%	Anonymous tracking in one site
Work 03 [3] Yang et al., 2022	Distributed multi-target association	Shared label and appearance table, no calibration	Benchmarks and an own mobile-camera set	MOTA, IDF1, FP/FN, ID switches	Real-time association without a server	No building-level state or visitor origin

Work and citation	**Problem or objective**	**Approach and contribution**	**Study context**	**Evaluation criteria**	**Key findings**	**Limitations and gap**
Work 04 [4] Casao et al., 2021	Distributed multi-target tracking	Consensus Kalman filter, re-ID and tracker manager	Public multi-camera datasets	MOTA/IDF1-type, bandwidth	Better than centralised in accuracy and bandwidth	One camera network; no occupants or visitors
Work 05 [5] Gaikwad and Karmakar, 2021	Edge multi-person multi-camera tracking	Detection, tracking and re-ID on Jetson TX2	Own campus recordings, one day	IDF1, FPS, re-ID latency	IDF1 90.97, about 30 FPS, about 90 ms	No multi-building state or query model
Work 06 [6] Mohan and Menon, 2020	Model camera-network identification and tracking	Abstract DSTS: states, events, transitions	Abstract framework	Analytical	Formal model with building-level BSTS	Visitors via a central master list
Work 07 [8] Nair et al., 2021	Face recognition and tracking	Recognition and tracking across cameras	Surveillance cameras	Recognition and tracking quality	Face is a workable identity cue	Not built around per-building galleries or access control
Work 08 [10] Rathgeb et al., 2014	Protect iris templates	Bloom-filter templates	Iris databases	Comparison and speed	Alignment-free, one-way templates	Iris, not face; one database, no routing
Work 09 [11] Marzani et al., 2026	Privacy-preserving crowd counting	Face features, fuzzy extractor, encrypted Bloom filters	Multi-location counting	Counting accuracy, privacy	Counts people without revealing faces	Counts, does not identify; no roles

Work and citation	Problem or objective	Approach and contribution	Study context	Evaluation criteria	Key findings	Limitations and gap
Work 10 [12, 16] Bloom, 1970; Kirsch and Mitzenmacher, 2006	Compact set membership	Bit array and hash functions; double hashing	Theory	False-positive rate, space	No false negatives; tunable error	Exact membership only; assumes independent items
Work 11 [13] Fan et al., 2000	Share web caches cheaply	Bloom-filter summaries of peers	Proxy traces	Messages, hit ratio	Far fewer inter-proxy messages	Exact keys, not similar faces
Work 12 [14, 15] Charikar, 2002; Lv et al., 2007	Similarity-preserving hashing	Random hyperplanes; multi-probe lookup	Theory, similarity search	Recall, tables needed	Angle-preserving codes; fewer tables	Generic; no privacy or access control

Chapter 4

Requirements

4.1 Functional requirements

Table 4.1 lists the functional requirements with their actors, inputs and outputs, and the acceptance criterion used to check each one. Table 4.2 then shows how the requirements combine into the main use cases.

Table 4.1: Functional requirements

ID	**Requirement**	**Actor**	**Input → output**	**Acceptance criterion**
FR1	Recognise a registered occupant against the building's own gallery	Camera event	Embedding → identity or abstain	Accepted when at least 12 of 20 references agree within 80.43°
FR2	Route an unrecognised capture to likely buildings with Bloom filters, then verify there	Building node	Embedding → shortlist, then identity	Shortlist of at most 5 buildings; only they receive the capture
FR3	Update the person's zone probabilities after each identified event	System	Identified event → nine zone probabilities	Nine rows per event summing to 1
FR4	Remember visitors for a day and promote frequent visitors	System	Arrivals → day cache, permanent pool	Repeat sightings need no routing; promotion after 3 entries in 7 days or 8 in 30
FR5	Open and close a visit pointer at the home building	System	Entry and exit events → pointer record	Pointer minted at arrival, closed at departure; no biometrics stored

ID	Requirement	Actor	Input → output	Acceptance criterion
FR6	Answer 13 query types and a full-trajectory query	Dean, Teacher, Student	Query → answer	Answer consistent with the stored events
FR7	Apply a two-layer policy before any answer	Query service	Caller, target, purpose → permit or deny	Every denied request returns a reason code and no data
FR8	Return an explanation with each answer, redacted to the caller's level	Query service	Answer → explanation	Explanation never shows more than the answer; faithful to it
FR9	Seal every cross-building message	Building node	Message → sealed envelope	Tampered, replayed or mis-addressed messages are dropped
FR10	Show the campus, zones, queries and security log on a dashboard	All roles	API data → web page	Content filtered by the caller's role
FR11	Compare with a centralised search and report TP, FP, FN, TN	Evaluator	Run files → metrics	Metrics per cohort, state level and seed
FR12	Detect a suspected mask and relax the local acceptance thresholds	Local recogniser	Angle matrix → adjusted thresholds	Applied to the local gallery check only; can be switched off
FR13	Record every authorization decision in an audit log	Access layer	Decision → log entry	Entry has layer, code, principal and target
FR14	Sign users in as a demo role or with a per-person credential	Users	Credential → signed token	Token expires in one hour; lockout after repeated failures

Table 4.2: Use cases

ID	Use case	Main flow	Requirements
UC1	Identify a registered occupant	Capture at the home building; local gallery accepts; state updated	FR1, FR3, FR12

ID	Use case	Main flow	Requirements
UC2	Identify a visitor for the first time today	Local check fails; Bloom routing ranks buildings; shortlisted buildings verify over sealed channel; pointer opened; references cached for the day	FR1, FR2, FR3, FR4, FR5, FR9
UC3	Recognise a returning visitor	Day cache or permanent pool matches locally; no request leaves the building	FR3, FR4
UC4	A Dean asks where a student of the campus was	Authenticated; Layer 1 and Layer 2 permit at L4; answer and explanation returned	FR6, FR7, FR8, FR13, FR14
UC5	A student asks about a peer	Layer 2 denies at L0; refusal with reason code; nothing disclosed	FR7, FR13
UC6	An administrator reviews the evidence behind an answer	"Why this answer" card shows rule, evidence, uncertainty and what was withheld	FR8, FR10
UC7	An evaluator compares the system with a centralised search	Run files scored per cohort and state level; five seeds averaged	FR11

4.2 Non-functional requirements

Table 4.3 states the quality requirements with the measure used. The result that satisfies each one is given in Chapter 6.

4.3 Hardware requirements

A laptop-class machine is sufficient: an Intel CPU, 16 GB of memory recommended, and no GPU. All face-model inference is done once, offline, on the CPU; the experiments then work on stored 512-dimensional embeddings. No cameras or dedicated edge devices are required because camera events are replayed from the datasets. A full one-day run of 11,000 events, including enrolment, routing, tracking and evaluation, takes about eight minutes on such a machine.

4.4 Software and data requirements

Table 4.4 lists the software, models and datasets, with their sources.

Table 4.3: Non-functional requirements

Quality	**Requirement**	**Measure**	**Where reported**
Accuracy	Decentralised identification close to a centralised search	Overall accuracy within 1 point; visitor F1 reported	Section 6.3.1
Communication cost	Touch a small part of the galleries per lookup	Gallery touched per lookup (central = 100%)	Section 6.3.1
Privacy	No gallery, id or embedding in a published filter; no plaintext biometric between buildings	Filter contents inspected; sealed-transfer tests; effective false-positive rate	Sections 6.3.8, 6.3.9
Security	Fail closed; resist tampering, replay, impersonation and brute force	Attack tests; lockout after 5 failures in 60 s	Section 6.3.9
Explainability	Explanations faithful to the answer, free of leakage, and informative about uncertainty	Faithfulness cases; flag precision and recall; latency overhead	Section 6.3.7
Reproducibility	Every result regenerable from a seed	Seeded populations; five seeds	Section 6.3.3
Performance	Run on a laptop CPU	11,000 events in about 8 minutes	Appendix A.1
Usability	One web page for all roles	Dashboard with role-filtered views	Section 6.3.10

Table 4.4: Software, tools and data

Item	**Details**
Operating system, language	Windows 11; Python 3.12 in a virtual environment
Numerical and data	NumPy, pandas, matplotlib; ONNX Runtime and OpenCV for the one-off embedding extraction
Face models	InsightFace buffalo_l: SCRFD detector (det_10g) [18] and ArcFace recogniser (w600k_r50) [17], 512-dimensional output
Routing and state	Random-hyperplane LSH [14], Bloom filter [12], vote-based verifier, SQLite state stores (written from scratch)
Security and API	cryptography library (X25519, HKDF, AES-GCM, Ed25519), HMAC tokens, FastAPI and uvicorn, Prometheus client
Front end	Static HTML and JavaScript dashboard served by the API (no build step)
Testing	pytest with 571 automated tests
Main dataset	CASIA-WebFace [19]: 500 identities, 20 reference and 20 test photos each (20,000 embeddings), in 10 buildings of 50 people
Masked dataset	RMFRD [20]: 50 identities as an 11th building; 20 unmasked references and 20 masked probes each
Access conditions	Both datasets are public research datasets; the RMFRD page states no licence beyond a request to cite it

Dataset partitioning. For each person, the 20 reference photos enrol them in their building and are never used as events; the 20 test photos are the camera events, so no photo is used for both. Buildings are formed by a seeded random permutation of the 500 people into 10 buildings of 50; the 50 masked people always form the 11th building and are not shuffled by the seed. The Bloom filters and the mean face used for centring come only from reference photos. No model is trained or tuned on the test photos.

Chapter 5

Proposed Methodology

5.1 Design (architecture) or conceptual framework

The environment is a set of buildings. Each *building node* owns its registered occupants' gallery, a state store with a registered and a visitor database, and a visitor lifecycle manager. A *secure communication layer* connects the nodes; an *access control layer* sits between the users and the data; and a *query service* applies the policy and builds the response with its explanation. Figure 5.1 shows the architecture and Figure 5.2 the two main workflows: the inter-building tracking workflow (recognise locally, route, verify, update the state) and the querying workflow (authenticate, authorise, retrieve, apply the response policy, explain).

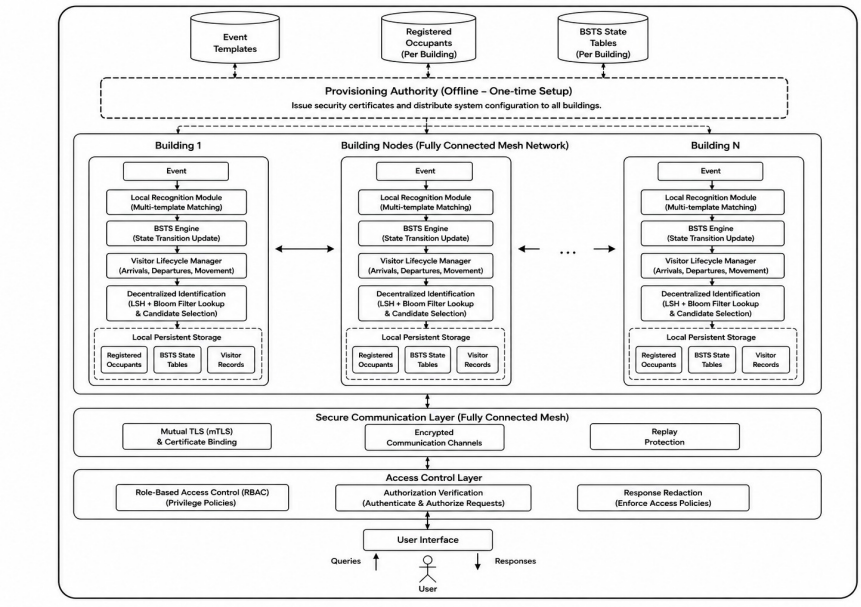


Figure 5.1: System architecture: building nodes, secure communication layer, access control layer and user interface

Table 5.1 lists the main components and the module of the implementation that realises each. Table 5.2 lists what each building keeps and what leaves it.

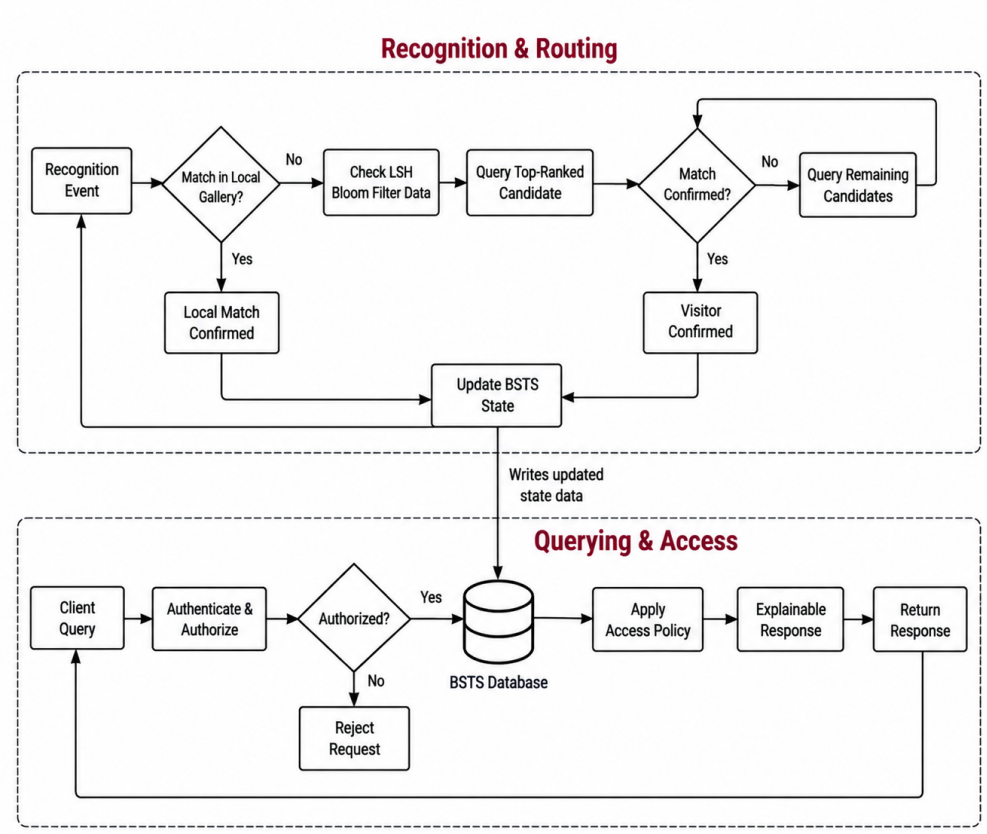


Figure 5.2: Inter-building tracking workflow and querying and access workflow

Table 5.1: Components and their responsibilities

Component	**Responsibility**	**Module**
Enrolment	Turn a building's 20 reference photos per occupant into centroids, LSH codes and a Bloom filter	buildinglib.enroll
Local recogniser	Vote over the building's own gallery; mask handling	pipeline.recognize, recognition
Router and verifier	Rank peer buildings by Bloom score; verify the capture in the shortlisted buildings	buildinglib.route, verify
State store (BSTS)	Per-occupant probability over the nine zones; registered and visitor databases	dsts.state
Visitor manager	Presence, day cache, entry history, permanent pool, visit pointers	buildinglib.visitor_pool
Secure transfer	Sealed, signed, replay-protected messages; certificates	secure_transfer
Access control	Tokens, credentials, lockout, two-layer policy, audit log	access_control
Query engine	Thirteen query templates and the trajectory query over the identified events	pipeline.query
Explanation layer	Rule, evidence, uncertainty flags, redaction	pipeline.explain
API and dashboard	FastAPI server and static web page	dsts.api
Evaluation	Event and state confusion metrics, centralised baseline, explanation and Bloom measurements	evaluation, centralized, tools

Table 5.2: What a building keeps and what it shares

Item	**Content**	**Leaves the building?**
Gallery	20 reference embeddings per registered occupant	Only the matched occupant's 20 references, sealed, after a confirmed match
Bloom filter	Bit array of LSH items, 58,469 bits (7.3 KB), 7 hash positions, no ids or embeddings	Yes, published to all peers; one-way
registered.db and visitor.db	Per-occupant zone probabilities and events, for own occupants and for visitors	No
pointers.db	Visit pointers of own occupants who are away: occupant id, current building, entry and exit times	Read by location queries; contains no biometrics
visitor_pool.db and permanent_visitors	Entry history and the embeddings of promoted frequent visitors	No
Day cache	A visitor's 20 references, in memory for one simulated day	No; zeroed at day rollover

Novelty. On the technical side, (a) each building publishes only a Bloom filter of LSH codes, so a capture can rank buildings without any gallery leaving them, and (b) only the shortlisted buildings verify the face. On the application side, (c) a two-layer, class-teacher access policy with graded disclosure, (d) an explanation that is built from the same events as the answer and redacted to the caller's level, and (e) sealed inter-building transfers with certificate-bound keys.

5.2 Proposed approach and procedure

5.2.1 Embedding preprocessing and enrolment

Faces are detected and aligned with SCRFD [18] and embedded with an ArcFace model [17] (512 dimensions). A raw embedding e is normalised, centred with the mean $\bar{x}$ of the reference embeddings, and normalised again:

$$\hat{e} = \frac{e/\|e\| - \bar{x}}{\| e/\|e\| - \bar{x} \|} \tag{5.1}$$

The $R = 20$ preprocessed references of an occupant u are averaged into a unit-norm centroid:

$$z_u = \frac{\sum_{r=1}^{R} \hat{e}_{u,r}}{\left\| \sum_{r=1}^{R} \hat{e}_{u,r} \right\|} \tag{5.2}$$

The centroid is used only to build the Bloom filter. The 20 individual references are kept in the building for voting, because a centroid alone is a poor verifier.

5.2.2 LSH codes and the Bloom filter

Random hyperplanes (one Gaussian vector w for each slot l and bit k, drawn once and shared by all buildings) define $L = 122$ slots of $K = 11$ bits [14]. The bit h is the side of the hyperplane on which the embedding lies, and the K bits of slot l form an integer code c_l:

$$h_{l,k}(\hat{e}) = \mathbf{1}\left[w_{l,k}^{\mathsf{T}} \hat{e} \geq 0 \right], \qquad c_l = \sum_{k=0}^{K-1} h_{l,k}\, 2^k \tag{5.3}$$

Each occupant's centroid yields L items, the item for slot l being $l \cdot 2^K + c_l$, so a code counts as a match only in the same slot. The items of all occupants of a building are inserted into a Bloom filter [12] with positions derived by double hashing [16]. The filter is sized for 50 occupants at a target false-positive rate of 1%, which gives 58,469 bits and 7 hash positions; with $50 \times 122 = 6{,}100$ items the fill ratio is about 51%. Only the packed bits and a few scalars are published (7.3 KB, or about 10 KB as a file): no ids, embeddings or centroids. Section 6.3.8 measures how close the real false-positive rate is to the 1% target.

5.2.3 Local recognition, routing and verification

A capture is first compared with the capturing building's own gallery. For every candidate occupant the angle between the capture and each of the 20 references is computed, and a reference votes if its angle is at most 80.43°. The winner is the occupant with most votes (ties broken by the smallest single angle); it is accepted only with at least 12 votes. Figure 5.2 gives the flow and the listing below the steps.

If the local check does not accept, the capture is hashed in all L slots. As well as the exact code, the 3 least confident bits of each slot are flipped to give extra probes [15]. Each peer's filter is tested; a building's score is the number of distinct slots that matched, on a 0 to L scale, and the top 5 buildings form the shortlist. They are contacted in rank order over the sealed channel and verify the capture with the same vote rule against their own occupants; the first confirmation ends the search. If no one confirms, the capture is not identified (abstain).

```
identify(capture q, building b):
  r = preprocess(q)
  (who, votes) = vote(own_gallery(b), r)          # >= 12 of 20 within 80.43
  if votes >= 12: return who                      # local occupant
  codes, margins = lsh_encode(r)                  # 122 slots of 11 bits
  score[c] = distinct slots of c's Bloom filter that match any probe
  for c in top_5(score):                          # best score first
      reply = sealed_request(c, r)                # only c sees the capture
      if reply.matched: return reply.occupant     # references come back sealed
  return None                                     # abstain
```

5.2.4 Mask-occlusion handling

The local recogniser contains a mask handler (module `recognition.mask_handler`) that was part of the pipeline before this evaluation and is *on by default*, so every result in Chapter 6 includes its effect. After the first vote pass it measures the standard deviation of the per-occupant minimum reference angles over the building's gallery. If this spread exceeds 6.5° it treats the capture as suspected to be masked, loosens the acceptance angle by 4° (from 80.43° to 84.43°), reduces the required votes by 25% (from 12 to 9), repeats the vote and records a mask flag on the result.

Two properties matter for the interpretation of the masked results. First, the handler acts only on the *local* check of the capturing building; the Bloom routing and the remote verification always use the plain thresholds, so a masked visitor is never helped by it. Second, the detector is a statistical heuristic on the angle matrix, not an image classifier: a large spread means that one person stands out clearly from the others in the gallery, which is the signature of a confident match rather than of a mask. Section 6.3.6 measures what it actually flags and what

happens when it is switched off with the environment variable `DSTS_MASK_HANDLER=off`, which was added for that ablation and leaves the default unchanged.

Table 5.3: Mask handler parameters (defaults)

Parameter	**Value**	**Effect**
Angle-spread threshold	6.5°	Above it, a mask is suspected
Acceptance-angle relaxation	+4°	80.43° becomes 84.43°
Vote relaxation	25% fewer	12 votes become 9
Minimum gallery size for detection	3 occupants	Below it the handler never fires
Where it is applied	Local check only	Not in routing or remote verification
Switch	`DSTS_MASK_HANDLER`	"off" disables it; default on

5.2.5 Tracking: probabilities and state

For an identified event with candidate distances d_i to each candidate occupant i, the occupant probabilities follow Definition 3.3 of the DSTS framework [6]: with σ the population standard deviation of the distances,

$$p_i = \frac{\exp\left(-d_i^2/2\sigma^2\right)}{\sum_l \exp\left(-d_l^2/2\sigma^2\right)} \tag{5.4}$$

The result is uniform over ties when $\sigma = 0$. The Bloom score, vote counts and ground truth never enter this probability. The building state store keeps, for each occupant, a probability over the nine zones and updates it from each event. The zones are z1 Entrance, z2 Mail room, z3 Office, z4 Lounge, z5 Conference, z6 Classroom, z7 Cafeteria (reached only through the exit), z8 Exit, and zT, the transition zone to the outdoors; movement between buildings must pass through zT. The event is stored in the building that observed it: registered occupants in the registered database and visitors in the visitor database. The state is reset at the start of every simulated day.

5.2.6 Dynamic visitor database and visit pointers

A visitor is remembered in three layers (Table 5.4). A *transition-zone rule* keeps a visitor who is waiting in zT out of other people's probability updates until they are seen outside zT. A *visit pointer* (occupant id, current building, entry and exit times, no biometrics) is minted at the home building when the visitor arrives and closed at departure, so a location query follows the pointer instead of asking all buildings.

An arrival is the zT event of a visitor who is not inside. When a visitor's *earlier* entries into a building already satisfy a rule from the configuration (default: three entries in seven days, or eight in thirty), the *next* entry stores their embeddings permanently. From then on the visitor

Table 5.4: The three layers of the visitor database

Layer	**Lifetime**	**Content**	**Used for**
Presence	Memory, until the visitor leaves or the day ends	Who is inside and in which zone	Which rows the probability update covers
Day cache	Memory, one simulated day; survives departure; zeroed at rollover	The visitor's 20 reference embeddings	Matching later captures the same day with no routing
Entry history and permanent pool	Disk: visitor_pool.db, permanent_visitors	Every arrival; embeddings of frequent visitors	Recognising frequent visitors on any later day without routing

is matched from the permanent pool, on every day, without contacting any other building. A wrong identity stored in the cache persists for the rest of the day; such cases are counted separately in the results.

5.2.7 Role-based access control

Access control follows the role-based model of Sandhu et al. [21], extended with the relationship between the caller and the target. Every query passes two layers before execution. *Layer 1* authenticates the principal and applies the building scope. *Layer 2* is the persona policy, which returns a disclosure level for the pair (caller, target) and the purpose and time of the query. The levels are shown in Table 5.5 and the access model in Table 5.6. There are 1 Dean, 10 Teachers and 39 Students per building, and each Teacher has a class roster of about four students.

Table 5.5: Disclosure levels

Level	**Name**	**What the caller sees**
L0	None	Nothing; the request is refused with a reason code
L1	Presence	Availability only (defined in the policy; no current rule grants it)
L2	Current zone, coarse	Functional sector and the time rounded down to the hour
L3	Precise current	Exact zone, zone label, confidence and exact time
L4	Historical track	Full trajectory and dwell times

Building-level queries (Q1, Q2) return aggregates to everyone in their own building and the list of people only to a Dean; zone queries (Q10, Q12, Q13) filter each listed occupant through the same policy; students cannot run zone queries. The templates are listed in Table 5.7.

Table 5.6: Access model (class-teacher RBAC)

Role	**May follow**	**Disclosure**	**Cannot see**
Dean (1 per building)	Teachers and students of their own campus (wherever they are), and the Deans of other buildings	L4 for both (rules RULE_A_OWN_CAMPUS_FULL and RULE_A_PEER_DEAN_TRACK)	Other campuses' teachers and students; visitors
Teacher (10 per building)	Only the students on their class roster, in any building	L4 for roster students (RULE_B_ROSTER_STUDENT)	Unassigned students, peer teachers, the Dean
Student (39 per building)	Themselves in full; the teachers assigned to them	L4 for self, L3 for assigned teachers (RULE_C_ASSIGNED_TEACHER)	Peer students, other teachers, the Dean

Table 5.7: Query templates

Query	**Question**
Q1	Did any occupant stay in building b after time t?
Q2	Was the number of visitors in building b more than the number of registered occupants at time t?
Q3	Did occupant o leave building b before time t?
Q4	Did occupant o enter the building between times t_1 and t_2?
Q5	Did occupant o visit all the zones in building b?
Q6	Where was occupant o at time t?
Q7	How many zones did occupant o visit during the interval t_1 to t_2?
Q8	At what time did occupant o visit zone z for the first time?
Q9	How long did occupant o spend at zone z during the first visit?
Q10	Who was in zone z at time t?
Q11	Where was occupant o during the interval t_1 to t_2?
Q12	Who was in zone z from time t_1 to t_2?
Q13	Who all visited zone z?
TRACK	Full historical trajectory of occupant o

5.2.8 Explainable responses

Each query builds an explanation from the same filtered event list that it used to compute the answer, never by recomputing afterwards. The explanation holds the decision rule in one sentence, the evidence events with their roles (decisive, last before t, carry-in state, zone visit), what was looked for and not found, uncertainty flags, what redaction withheld, and the visit pointer if one answered the query. Flags come from the identification provenance stored with every identified event (votes, routing rank, Bloom margin, whether the vote winner was also the nearest candidate) and are listed in Table 5.8. The identity probability of Equation 5.4 is not used as a confidence signal because it is close to 1 for most misidentifications too. Before leaving the service the explanation is redacted to the caller's disclosure level, so it never shows more than the answer does.

Table 5.8: Uncertainty flags

Flag	**Raised when**
vote_winner_not_nearest	The accepted occupant was not also the nearest candidate (strongest error signal)
low_votes	Fewer than 16 votes, although 12 suffice to accept
verified_by_lower_ranked_ building	The confirming building was not first on the shortlist
narrow_routing_margin	The Bloom score gap between the top two buildings is below 5
stale_observation	A point-in-time answer rests on a sighting more than 60 minutes old
pointer_based	A value comes from a visit pointer with no identified sighting behind it

5.2.9 Secure inter-building exchange

Every message between buildings is sealed (Table 5.9). The sender generates an ephemeral X25519 key [23], derives a symmetric key with HKDF-SHA256 [22], encrypts with AES-GCM [25] using the whole header as associated data, and signs the header, nonce and ciphertext with Ed25519 [24]. Replies are bound to their request. A campus certificate authority issues node certificates that bind both the signing key and the key-agreement key; expiry, revocation, subject and pinning are checked. A replay guard accepts a message only within a 300-second window and once per nonce, and runs after the signature check so the timestamp is authenticated. Any failed step yields no reply and nothing is admitted (fail closed).

5.2.10 The masked building

To study masked faces, 50 more people from RMFRD [20] form an 11th building. The 50 people with the most real masked photos are used. Each has 20 distinct real unmasked photos as the gallery and 20 masked probes: all their real masked photos up to 20 (800 of the 1,000

Table 5.9: Security mechanisms

Mechanism	**What it does**	**Module**
Sealed payloads	Ephemeral X25519, HKDF-SHA256, AES-GCM with the header as associated data; Ed25519 signature; reply bound to request	secure_transfer.payload
Certificates	CA-signed, binding both public keys; expiry, revocation, subject and pinning checked	secure_transfer.transport
Replay guard	300 s window plus unique nonces, after signature verification	secure_transfer.replay_guard
Fail closed	Any failed step: no reply, nothing admitted	secure_transfer.handler
Signed tokens	HMAC-SHA256, one-hour expiry, mintable only by the server	access_control.tokens
Per-person credentials	Credential is HMAC of the occupant id under a master secret; one never opens another account	access_control.credentials
Login lockout	5 failures in 60 s per client and occupant, or 20 in 5 min per occupant, then HTTP 429	access_control.rate_limit
WebSocket tickets	Single-use ticket valid 30 s; the token is never put in a URL	access_control.tickets
Audit log	Every decision with its layer and code	access_control.authorize
Visitor data hygiene	Private copies; day cache zeroed at rollover	buildinglib.visitor_pool

probes), topped up with augmented copies (flip, rotation, scale, shift, brightness, blur, noise, JPEG compression) for the 200 remaining. Photos are detected and aligned with SCRFD and embedded with the same ArcFace model. Routing and verification are unchanged: a masked probe is matched against an unmasked gallery. The masked people are not shuffled by the seed. An earlier version built from the MFR2 dataset gave misleadingly high masked accuracy because each person had only one or two unmasked photos and the masks were partly synthetic; it was replaced by RMFRD.

5.2.11 Parameters

Table 5.10 lists the parameters used. The Bloom filter was configured for a 1% false-positive rate, but Section 6.3.8 shows that the effective rate that routing sees is about 3.5% because LSH codes of different people collide.

5.2.12 API and dashboard

The FastAPI server exposes the endpoints in Table 5.11; every data endpoint runs the same authentication, building scope and two-layer policy before it returns anything, and the dashboard is a static page that calls them.

Table 5.10: Parameters used

Parameter	**Value**
LSH bits per slot K / slots L	11 / 122
Bloom target false-positive rate (configured)	0.01
Bloom effective false-positive rate (measured, Section 6.3.8)	about 3.5%
References per occupant R	20
Accept angle / minimum votes	80.43° / 12
Shortlist size / extra probes	5 / 3
Embedding dimension	512
Mask handler (default on)	spread 6.5°; +4°; 25% fewer votes
Visitor promotion rule	3 entries in 7 days or 8 entries in 30 days
Replay window / token lifetime	300 s / 1 h
Login lockout	5 failures in 60 s per client and occupant, or 20 in 5 min per occupant
Uncertainty-flag thresholds	low votes below 16; narrow margin below 5; stale after 60 min
Contract seed	42

Table 5.11: Main API endpoints

Endpoint	**Purpose**
POST /api/auth/login	Demo persona (when enabled) or occupant id with that occupant's own credential; lockout after repeated failures
POST /api/ws-ticket	Single-use ticket for the live WebSocket
GET /api/layout, /api/occupancy, /api/heatmap	Buildings, who is where, and zone counts, each filtered by the caller's role
POST /api/query	Run one of the query templates under the two-layer policy; returns answer, decision and explanation
GET /api/security-log	The caller's own authorization decisions
POST /api/simulate, /api/demo-walk, /api/reset	Demo helpers: generate events, walk a person between buildings, reset
GET /metrics	Prometheus counters; needs a token or a loopback client

Chapter 6

Results and Analysis

6.1 Evaluation setup

The evaluation uses 550 occupants in 11 buildings: the 500 CASIA-WebFace people dealt into 10 buildings of 50 by a seeded permutation, and the 50 masked people in building 11. One simulated day has 11,000 events, one for each of the 11,000 test photos; 8,800 are local (the person is in their home building) and 2,200 are visitor events. Every headline number in this chapter is the *average over five random populations* (seeds 0 to 4); each seed places the same people in different buildings and draws a different event day. Where a single run is needed to show detail (the visitor outcome buckets, the mask-handler ablation and the screenshots), seed 42 is used and is named as such. Table 6.1 summarises the protocol.

Table 6.1: Evaluation protocol

Item	**Setting**
Population	550 occupants: 500 (CASIA-WebFace) in 10 buildings of 50, plus 50 masked people (RMFRD) in building 11
Events per run	11,000 (8,800 local, 2,200 visitor); the 20 test photos of every person, one simulated day
Seeds	0, 1, 2, 3, 4 for all averages; 42 for the detailed run
Baseline	Hypothetical centralised server holding every gallery, same preprocessing and vote rule, no LSH or Bloom filters
Ground truth	True occupant and place of every event; a full true snapshot of the campus after every event
Hardware	Laptop CPU, no GPU

Safeguards against bias. Reference photos enrol people and are never events; test photos are events and are never enrolled. The mean face and the Bloom filters use reference photos only. The centralised baseline uses exactly the same preprocessing, thresholds and vote rule as the building nodes, so any difference is caused by routing and not by a stronger matcher. Ground truth is read only by the evaluation scripts; recognition, routing and tracking receive

observable event fields only.

6.2 Evaluation criteria

All identification metrics use the one-vs-rest convention. An item has one true label and at most one predicted label out of N_L labels. **TP**: the predicted label is the true label. **FP**: a wrong label was predicted. **FN**: the true label was not predicted (a wrong prediction or an abstention). **TN**: every other (item, label) pair, that is items $\times N_L - \text{TP} - \text{FP} - \text{FN}$. A wrong identity is therefore both an FP and an FN, and an abstention only an FN.

$$\text{Precision} = \frac{TP}{TP+FP}, \qquad \text{Recall} = \frac{TP}{TP+FN}, \qquad F_1 = \frac{2PR}{P+R} \tag{6.1}$$

Recall equals the share of events identified correctly, so it is also called identification accuracy. Because TN is in the millions, accuracy computed with TN is close to 1 and is not used to compare systems.

Event level. The label is the occupant id (550 labels), reported for all, local, visitor, masked and unmasked events. **State level.** State s_k is the campus after event k; the label is (building, zone) with 99 labels, plus building (11 labels) and zone (9 labels) alone, and every occupant the system places counts, including visitors updated by other people's events. **Cost.** Gallery touched per lookup is the number of buildings whose gallery is searched divided by the number of buildings (the centralised server is 100%); routing recall is the share of routed visitors whose home building is in the shortlist.

Explanation criteria. A flag is useful if it is carried by wrong answers and not by right ones. For identifications, *share of wrong flagged* is the fraction of wrong identifications that carry at least one flag, and *share of correct flagged* the false-alarm rate. For answers, an answer is wrong when it differs from the same query run over the identified events with their true occupants; flag *precision* is the share of flagged answers that are wrong and *recall* the share of wrong answers that are flagged. Overhead is the extra latency and response size of building the explanation. **Bloom criterion.** The effective false-positive rate is the share of the enrolled items of the other buildings that a building's filter reports as present.

6.3 Result

6.3.1 Identification: decentralised against centralised

Table 6.2 compares the decentralised pipeline with the centralised search, and Table 6.3 gives the confusion counts and scores, all averaged over the five seeds. Figure 6.1 shows the accuracy by cohort.

Over all events the decentralised pipeline identifies 91.54% against 92.24% for the central

Table 6.2: Identification accuracy and gallery touched (average of five seeds, 11,000 events per run)

Cohort	**Events**	**Centralised accuracy**	**Decentralised accuracy**	**Gap (points)**	**Gallery touched**
All events	11,000	92.24%	91.54%	0.70	10.3%
Local	8,800	91.98%	92.02%	-0.04	9.1%
Visitor	2,200	93.27%	89.61%	3.66	15.1%
Unmasked people	10,000	95.48%	94.78%	0.70	10.1%
Masked people	1,000	59.80%	59.06%	0.74	12.6%

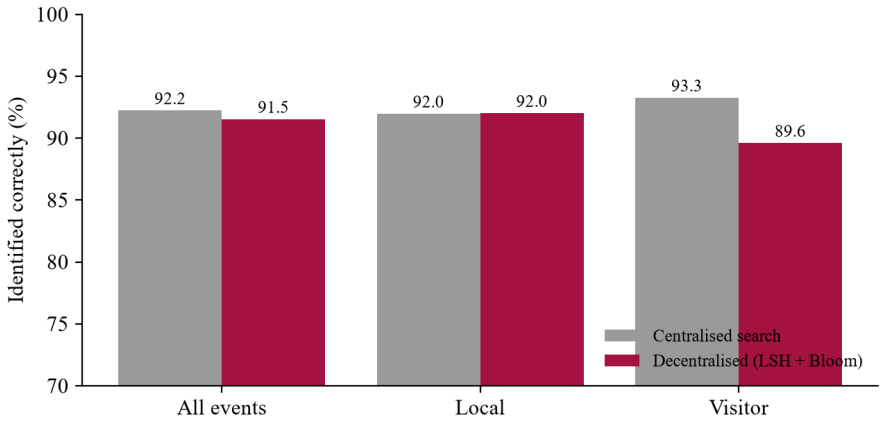


Figure 6.1: Identification accuracy, centralised search against the decentralised pipeline (average of five seeds)

Table 6.3: Event-level confusion counts and scores (average of five seeds; TN, in the millions, omitted)

Cohort	**TP**	**FP**	**FN**	**Precision**	**Recall**	**F1**
All events	10,069	245	931	0.976	0.915	0.945
Local	8,098	154	702	0.981	0.920	0.950
Visitor	1,971	91	229	0.956	0.896	0.925
Unmasked	9,478	173	522	0.982	0.948	0.965
Masked	591	72	409	0.892	0.591	0.711

search, a difference of 0.70 points, while each lookup touches 10.3% of the galleries. Local events are marginally better with the pipeline (92.02% against 91.98%) because a person is matched only among the 50 people of their own building, so there are fewer look-alikes to confuse them with; the central search must discriminate among 550. The visitor events lose 3.7 points (89.61% against 93.27%), which is the cost of routing. The standard deviation over the five seeds is only 0.16 points (Section 6.3.3).

6.3.2 Routing and the visitor database

Table 6.4 summarises how visitors are identified and Figure 6.2 breaks one run (seed 42) down by outcome. In that run, of the 2,200 visitor events, 1,447 (about two thirds) are served from the building's visitor database (the same visitor sighted again the same day), 708 are routed, and the remaining 45 are visitors that an occupant of the capturing building wrongly accepted as a local person (a cross-population collision). The three groups add up to 2,200. Errors among the routed events are mostly routing misses that end in an abstention, and a smaller number of cases in which the right building is reached but the face still does not pass verification.

Table 6.4: Visitor identification and tracking (average of five seeds)

Metric	**Value**
Visitor events per run	2,200
Precision / recall / F1 of visitor identification	0.956 / 0.896 / 0.925
Routing recall (home building in the top five)	86.9%
Buildings contacted per routed lookup	2.07
Buildings contacted per visitor event, counting database hits	0.67
Visitor events served from the visitor database	1,449 (about two thirds)
Visit-pointer precision	97.8%
Visitor recall in the tracked state: building / zone / building and zone	0.945 / 0.924 / 0.891

The detailed run also shows how the headline gap should be read. The centralised search identified 92.24% of its 11,000 events and the pipeline 91.76%. The exact difference is 52 events out of 11,000, that is 0.47 points; the two rounded percentages differ by 0.48 only because of rounding. The five-seed gap in Table 6.2 is 0.70 points.

6.3.3 Stability across five random populations

Table 6.5 lists each of the five populations. The averages in this chapter are the means of these rows; the spread is small, so the detailed run is representative and does not depend on one lucky assignment of people to buildings. The visitor result varies most because it rests

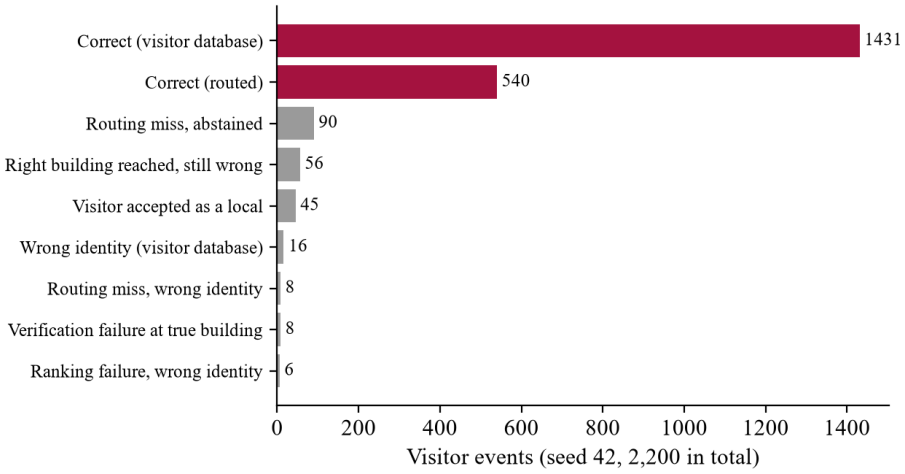


Figure 6.2: Outcome of the 2,200 visitor events in the detailed run (seed 42)

on fewer events.

Table 6.5: Results per random population (11,000 events each)

Seed	**Accuracy, all**	**Local**	**Visitor**	**Routing recall**	**Masked recall**
0	91.76%	92.11%	90.36%	87.1%	59.4%
1	91.35%	91.75%	89.73%	87.0%	59.3%
2	91.62%	92.08%	89.77%	86.4%	58.9%
3	91.44%	91.97%	89.32%	87.1%	59.1%
4	91.52%	92.18%	88.86%	86.6%	58.6%
Mean	91.54%	92.02%	89.61%	86.9%	59.1%
95% CI (± points)	0.20	0.21	0.69	0.4	0.4

6.3.4 Tracking state against the state ground truth

Table 6.6 reports the state-level scores. After every one of the 11,000 events the whole campus state, that is who is in which building and zone, is compared with the true state recorded by the simulation, and the scores are averaged over the 11,000 states of a run and then over the five seeds.

Table 6.6: State-level scores against the state ground truth (average of five seeds)

Level	**Labels**	**Precision**	**Recall**	**F1**	**Visitor recall**
Building	11	0.956	0.944	0.950	0.945
Zone	9	0.921	0.910	0.916	0.924
Building and zone	99	0.901	0.890	0.896	0.891

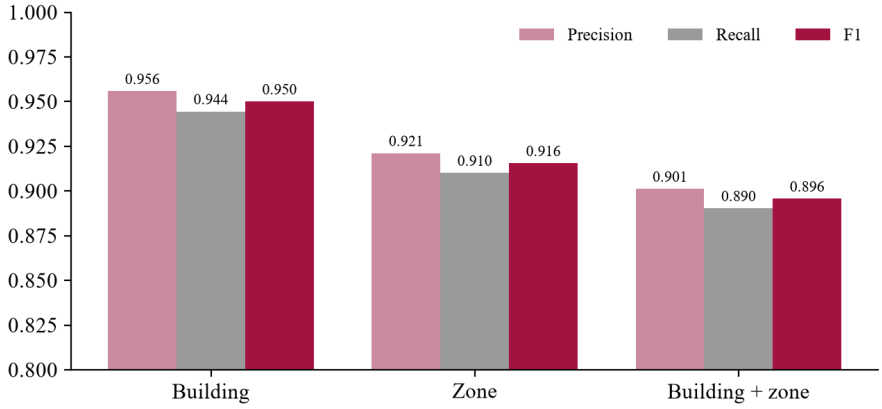


Figure 6.3: Precision, recall and F1 of the tracked state at three levels (average of five seeds)

Building-level state, that is knowing which building each person is in after every event, reaches an F1 of 0.950; the zone within a building and the combination are harder because the nine-zone belief is updated from one event at a time. The standard deviation of the building-level F1 over the five seeds is 0.0040.

6.3.5 Masked people

Table 6.7 compares the 50 masked people of building 11 (1,000 events per run) with everyone else (10,000 events) and Figure 6.4 shows the recall. Masked people are identified 59.1% of the time against 94.8% for unmasked people. When the system does answer for a masked person it is right 89.2% of the time, so the loss is mostly missed identifications rather than wrong ones. A masked probe is about 72° from its own unmasked gallery on average, close to its nearest other person (about 74°), so the gallery gives little margin. Routing is also weaker for masked visitors: the right building is found less often and ranked lower.

Table 6.7: Masked against unmasked people (average of five seeds)

Group	**TP**	**Precision**	**Recall**	**F1**
Unmasked people	9,478	0.982	0.948	0.965
Masked people	591	0.892	0.591	0.711
Masked, in their building	507	0.890	0.597	0.714
Masked, visiting	83	0.903	0.556	0.688

6.3.6 What was tried for masked faces, and the mask handler

Three ideas for improving masked recognition were tested offline on the 1,000 masked probes against the 50-person gallery, using the real thresholds (80.43° and 12 votes): adding ten synthetic-masked templates per person to the gallery; matching on the upper face only, by blanking the lower part of both gallery and probe; and relaxing the accept angle and votes when a mask is suspected. Table 6.8 and Figure 6.5 show that each mostly traded absten-

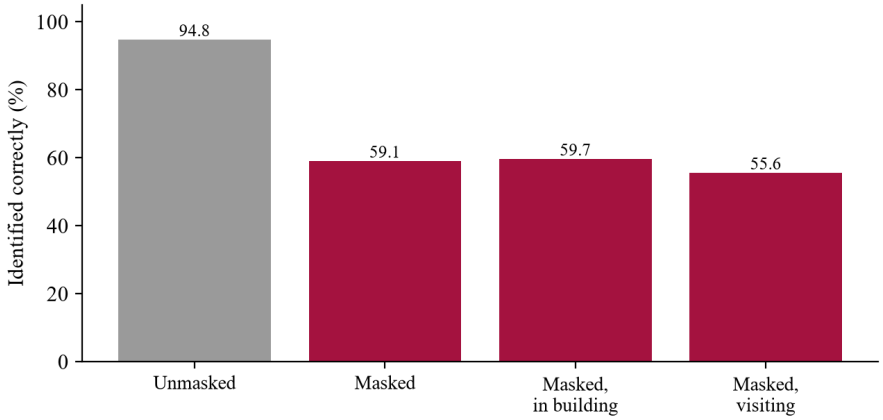


Figure 6.4: Share of events identified correctly, masked against unmasked people (average of five seeds)

tions for wrong answers. Relaxing the thresholds alone raised wrong identities from 7.0% to 28.4% and let strangers (people of other buildings) be accepted 31% of the time. F1 stayed at 0.72 or fell. Without any threshold, the nearest person is the right one only 65–70% of the time whatever the gallery (Table 6.9), so the limit is the face model, which was not trained for masks, and the noisy web labels. Enrolling five to eight *real* masked photos per person, held out from the probes, gave about five points more in rank-1 accuracy.

Table 6.8: Masked probes against the 50-person gallery: fractions of the 1,000 probes (offline, building 11)

Variant	**Correct**	**Wrong**	**Abstain**	**Strangers accepted**
Current: 20 unmasked references, plain thresholds	0.598	0.070	0.332	0.03%
Relaxed thresholds (+4°, 9 votes)	0.647	0.284	0.069	31.2%
Relaxed more (+8°, 9 votes)	0.556	0.444	0.000	100%
10 unmasked + 10 synthetic-masked references	0.627	0.118	0.255	0.00%
Same, with 9 votes	0.661	0.169	0.170	0.00%
Upper face only (gallery and probe)	0.572	0.297	0.131	0.00%
Upper face only, with 9 votes	0.595	0.349	0.056	0.07%

The existing mask handler. The handler described in Section 5.2.4 flags a probe as masked when the spread of per-person minimum angles is large. Measured on the detailed run it flags none of the 850 masked local events and 3,355 of the 7,950 unmasked local events (42%), because a large spread means that one person stands out, not that a mask is present. With the handler switched off (`DSTS_MASK_HANDLER=off`) on the same events, masked results are unchanged (590 TP, 70 FP) and unmasked people gain 21 correct identifications (9,525 against 9,504) and lose 22 false accepts (125 against 147). The default is left on, so all other

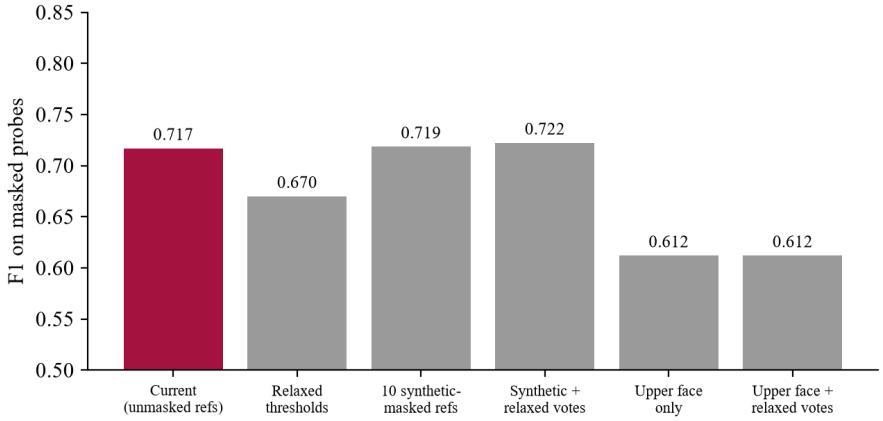


Figure 6.5: F1 of masked probes for the tested variants (offline, building 11)

Table 6.9: Rank-1 accuracy of masked probes without thresholds (50-way)

Gallery	**Best single reference**	**Mean of top 3**	**Mean of all**	**Centroid**
20 unmasked references	0.647	0.694	0.694	0.700
10 unmasked + 10 synthetic-masked	0.612	0.668	0.698	0.699
Upper face only	0.613	0.652	0.642	0.659

results include this small effect, and the finding is reported as a limitation.

6.3.7 Explanation quality

Explanation quality was measured in three ways: whether explanations are faithful to the answers, whether the uncertainty flags catch real errors, and what the explanations cost. All numbers below are averages over the five seeds and were produced on the current system (11 buildings, the masked building included) by `evaluation.evaluate_explanations`.

Faithfulness and leakage. Because every explanation is built from the events that produced the answer, faithfulness is checked by tests: for each query class there are cases with a yes answer, a no answer and an empty answer (40 cases in all), and each case is checked twice, once that every evidence item is one of the engine's own events and once that the answer recomputed from the evidence equals the engine's answer. Further tests check that no explanation reveals more than its answer at any disclosure level, that routing details are hidden below L4, and that cached explanations are never shared across levels. All pass (Section 6.3.9).

Do the flags catch errors? Table 6.10 scores the identification flags against the ground truth. On average 10,314 events per run are identified, of which 245 are wrong. 74.2% of the wrong identifications carry at least one flag, and 3.8% of the correct ones do (the false-alarm rate); of all flagged events 32.3% are wrong. The flags work best on routed visitors (87% of wrong ones flagged) and on local identifications (74%). They almost never flag a

wrong identification that comes from the visitor database (3%), because that path re-matches against cached references and carries little provenance; these are the errors that persist for the rest of a day, and are the main weakness of the explanation layer. The strongest single flag is `vote_winner_not_nearest`: 66% of the identifications carrying it are wrong.

Table 6.10: Uncertainty flags on identifications (average of five seeds)

Group	**Identified**	**Wrong**	**Wrong, flagged**	**Correct, flagged**	**Flagged that are wrong**
All identified events	10,314	245	74.2%	3.8%	32.3%
Local	8,259	161	73.7%	2.4%	38.0%
Routed visitors	613	72	86.8%	34.6%	25.1%
Visitor database	1,442	12	2.8%	0.01%	67%

Table 6.11: Individual flags on identifications (average of five seeds)

Flag	**Events carrying it**	**Of which wrong**	**Share wrong**
vote_winner_not_nearest	99	66	66.1%
low_votes	306	115	37.4%
verified_by_lower_ranked_building	129	38	29.7%
narrow_routing_margin	227	50	22.1%

At the level of answers, the flags are scored over a grid of real queries (Q6 at every hour, Q7 over four two-hour windows and TRACK, for every identified occupant). On average 7,589 answers per run were scored; 24.6% carry a flag; 912 are wrong, of which 80.2% are flagged (recall) and 39.2% of the flagged answers are wrong (precision). Table 6.12 gives the figures per query type. A flag is therefore a good reason to look twice, not a verdict: most flagged answers are still right, but four of every five wrong answers are flagged.

Table 6.12: Uncertainty flags on answers, per query type (average of five seeds)

Query	**Answers scored**	**Share flagged**	**Flag precision**	**Flag recall**
Q6 (where at time t)	4,858	22.5%	43.8%	89.2%
Q7 (zones in interval)	2,183	22.0%	14.8%	59.8%
TRACK (trajectory)	548	53.2%	62.1%	70.8%
All scored answers	7,589	24.6%	39.2%	80.2%

What do explanations cost? Each query type was run 100 times per seed with explanations off and on (Table 6.13, Figure 6.6). Summed over all fourteen queries the latency rises from 16.3 ms to 17.4 ms, an increase of 8.0%. The aggregate queries (Q1, Q13) cost most because they gather one evidence item per person; the point queries cost almost nothing.

The response grows more than the latency: with the full L4 explanation, the sum of the response sizes rises from about 15,820 to 117,290 bytes, because the evidence is listed. Lower disclosure levels receive much less.

Table 6.13: Cost of explanations per query (average of five seeds; 100 runs each)

Query	**Latency without (ms)**	**Latency with (ms)**	**Overhead**	**Bytes without**	**Bytes with (L4)**
Q1	0.48	0.65	38%	4,735	21,185
Q2	1.75	1.97	14%	235	11,483
Q3	0.62	0.63	3%	128	969
Q4	0.36	0.37	2%	241	1,086
Q5	0.37	0.39	6%	193	1,802
Q6	1.93	1.95	1%	531	1,071
Q7	0.34	0.39	15%	146	3,495
Q8	0.39	0.39	0%	195	776
Q9	0.40	0.41	2%	354	1,374
Q10	2.65	2.78	6%	1,225	6,142
Q11	0.41	0.44	7%	1,913	5,168
Q12	3.18	3.37	8%	1,326	25,014
Q13	1.16	1.33	25%	1,863	28,769
TRACK	2.24	2.32	4%	2,734	8,956

An earlier measurement of the same quantities on the 10-building system, taken before the masked building was added, found a higher flag recall (84% of wrong identifications flagged against 74% here) and a lower false-alarm rate (2.3% against 3.8%). Splitting the present measurement by cohort explains the difference. For unmasked people 80.6% of the wrong identifications are flagged and 2.3% of the correct ones, close to the earlier figures. For the masked people only 58.7% of the wrong identifications are flagged, while 27.5% of their correct identifications are flagged as well: masked captures have low votes and narrow margins, so the flags fire on many right answers and miss many wrong ones, and they are far less discriminating there.

6.3.8 The Bloom filters' real false-positive rate

Each filter is sized for a 1% false-positive rate on the assumption that its items behave like independent random keys. LSH codes of different people are not independent: two occupants whose embeddings fall on the same side of the same hyperplanes receive the same code in that slot. The effective rate was therefore measured directly, by testing the enrolled items of the ten other buildings against each building's filter and averaging over the buildings and the five seeds. Table 6.14 and Figure 6.7 give the result.

The effective rate is about 3.5 times the configured one. The Bloom array itself is not at fault:

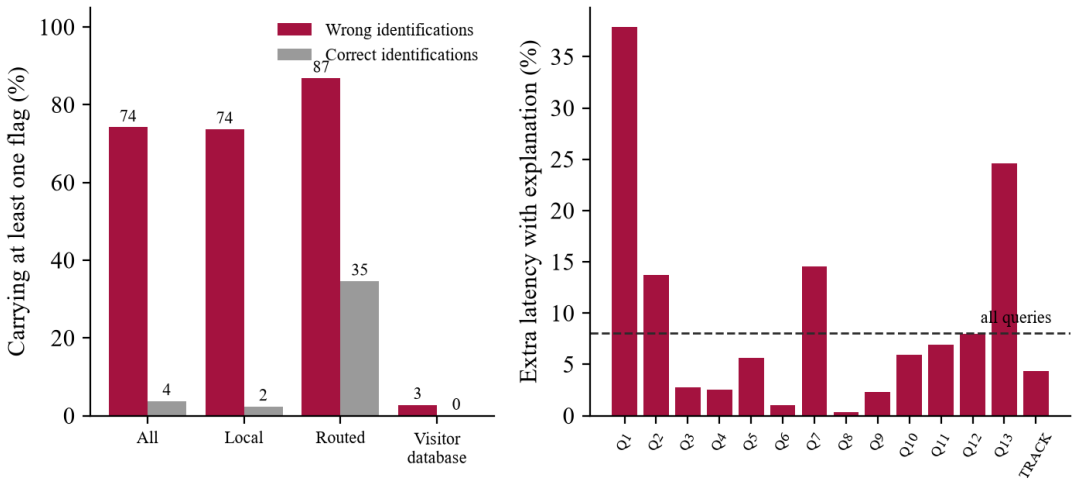


Figure 6.6: Left: share of wrong and of correct identifications carrying a flag. Right: extra latency of building the explanation, per query type (average of five seeds)

Table 6.14: False-positive rates of the building filters (average of five seeds)

Quantity	**Rate**	**Meaning**
Configured target	1.00%	Design value used to size the filter
Random integers never inserted (Bloom array alone)	0.95%	The array behaves as designed
Slot collisions expected if codes were uniform over 2048 values	2.44%	50 occupants over 2^{11} codes
Slot collisions measured (other buildings' items equal to an enrolled item in the same slot)	2.63%	LSH codes of different people collide
Effective rate seen by routing	**3.55%**	Items of other buildings reported as present

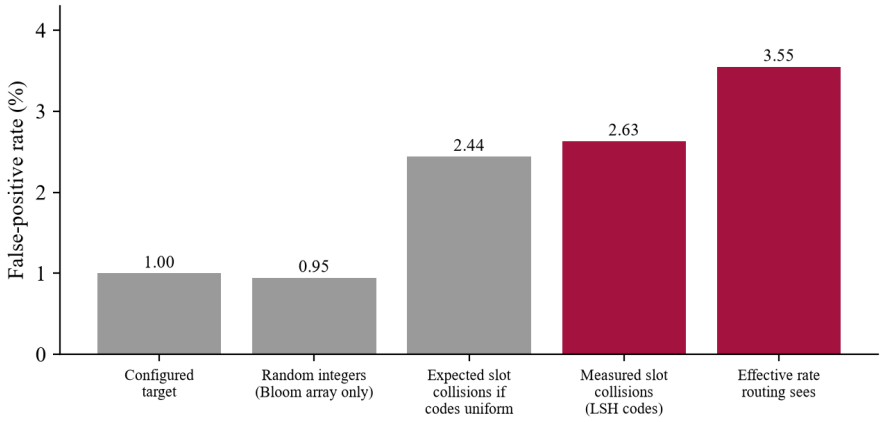


Figure 6.7: Configured and measured false-positive rates of the building filters

random integers that were never inserted are reported at 0.95%, essentially the 1% target. The excess comes from the codes: 2.63% of the items of other buildings already equal an enrolled item in the same slot, close to the 2.44% expected if the codes of 50 occupants were spread evenly over 2048 values. This is the same finding as in the earlier routing study [28], which measured $3.54 \pm 0.02\%$ on 10 buildings; the five-seed value here, measured on 11 buildings, is 3.55%.

Privacy consequence. The filter is one-way, but membership in it can be tested, and a positive does not prove that a particular face is enrolled in the building: with an effective rate of about 3.5% on top of genuine matches, a positive is ambiguous. That ambiguity is useful, and it is why routing only ranks buildings and never identifies anyone; identification needs the exact vote on the real references in the shortlisted buildings. The same finding also means that the configured 1% must not be quoted as the system's false-positive rate, and that the privacy argument should rest on the one-way encoding and on the verification step, not on a small false-positive rate. No formal privacy guarantee is claimed.

6.3.9 Verification of the security and policy layers

The security, policy and explanation layers are verified by 571 automated tests (Table 6.15). They include attack cases written for this project: tampering with every header field of a sealed message, replay, wrong sender or recipient, a revoked certificate, brute-force login, a token used as another person, and cache sharing across roles.

Table 6.15: Automated tests by area

Area	**Tests**	**What is checked**
access_control	112	Every policy cell, structured Layer-1 decisions, building scope, lockout, credentials, tickets, API-level RBAC attacks
secure_transfer	61	Sealed payloads: tampering, replay, wrong sender or recipient, revoked certificate, fail closed, strict decoding
api	92	Login, queries per role, caching, zone and time-frame views, dashboard adapter
queries	184	Query engine, multi-day normalisation, explanation faithfulness, leakage and uncertainty, CLI access rules
pipeline, population, events, evaluation	98	Phase 2, 3 and 4, visitor database, seeded populations, event generation, ground truth, confusion metrics
standalone, centralised, state library	19	Standalone demo, centralised baseline, state library
shared utilities	5	Dated timestamps, time-window arithmetic
Total	**571**	All passing on the final commit

A persona matrix tool replays 64 requests (Dean, Teacher, Student and an outsider against each query type and target type) and checks the decision, the rule code and the disclosure

level of each. Table 6.16 shows representative rows. A request for a person in another building is denied unless a relationship allows it: a Teacher may follow a rostered student wherever the student is, and a Dean may follow the Deans of other buildings; every other cross-building request is refused with a rule code such as `RULE_C_CROSS_BUILDING_DENIED` or `RULE_A_CROSS_BUILDING_B_C_DENIED`.

Table 6.16: Representative decisions from the persona matrix

Caller	**Target**	**Rule**	**Level**
Dean	Student of own campus	RULE_A_OWN_CAMPUS_FULL	L4
Dean	Dean of another building	RULE_A_PEER_DEAN_TRACK	L4
Dean	Student of another campus	RULE_A_CROSS_BUILDING_B_C_DENIED	L0
Teacher	Student on the roster	RULE_B_ROSTER_STUDENT	L4
Teacher	Student not on the roster, or the Dean	RULE_B_UNASSIGNED_DENIED	L0
Student	Assigned teacher	RULE_C_ASSIGNED_TEACHER	L3
Student	Peer student	RULE_C_PEER_STALKING_DENIED	L0
Student	Zone listing	RULE_C_PEER_ZONE_SURVEILLANCE_DENIED	L0
Student of another building	Dean	RULE_C_CROSS_BUILDING_DENIED	L0
Any person	Themselves	RULE_SELF_LOOKUP	L4
Anyone in the building	Building aggregate (Q1, Q2)	RULE_BUILDING_AGGREGATE	L2

6.3.10 The dashboard

Figures 6.8, 6.9 and 6.10 show the dashboard on the detailed run. The campus overview marks building 11 as the all-masked building. A Dean's query returns the answer with its "Why this answer" card; the same page for a Teacher returns a reduced answer, and the card states what was withheld and why.

6.4 Comparative Analysis

Table 6.17 compares the approaches. The fair comparison for this project is the same pipeline with a central search; other published systems [1, 2, 3, 4, 5] use different data and tasks and cannot be compared number for number.

Routing costs 0.70 points overall and 3.7 points on visitors, in exchange for touching about a tenth of the gallery and never pooling it.

Relation to the earlier routing study. The manuscript on routing alone [28] reports 92.81%

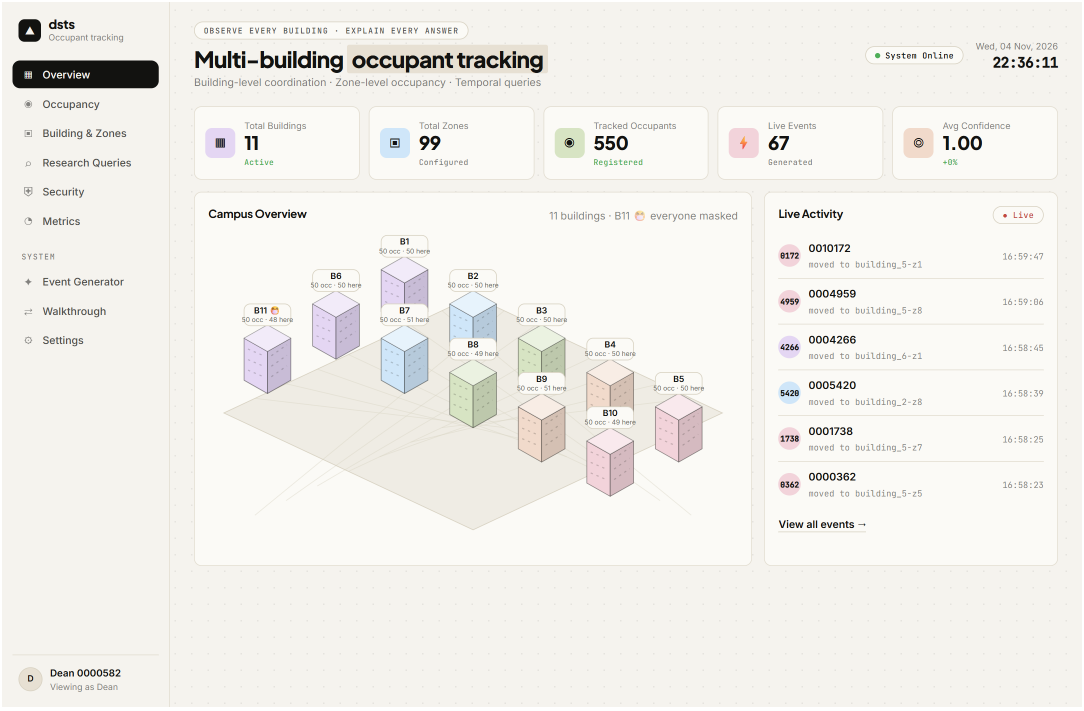


Figure 6.8: Campus overview: 11 buildings and 550 occupants, with building 11 marked as the all-masked building

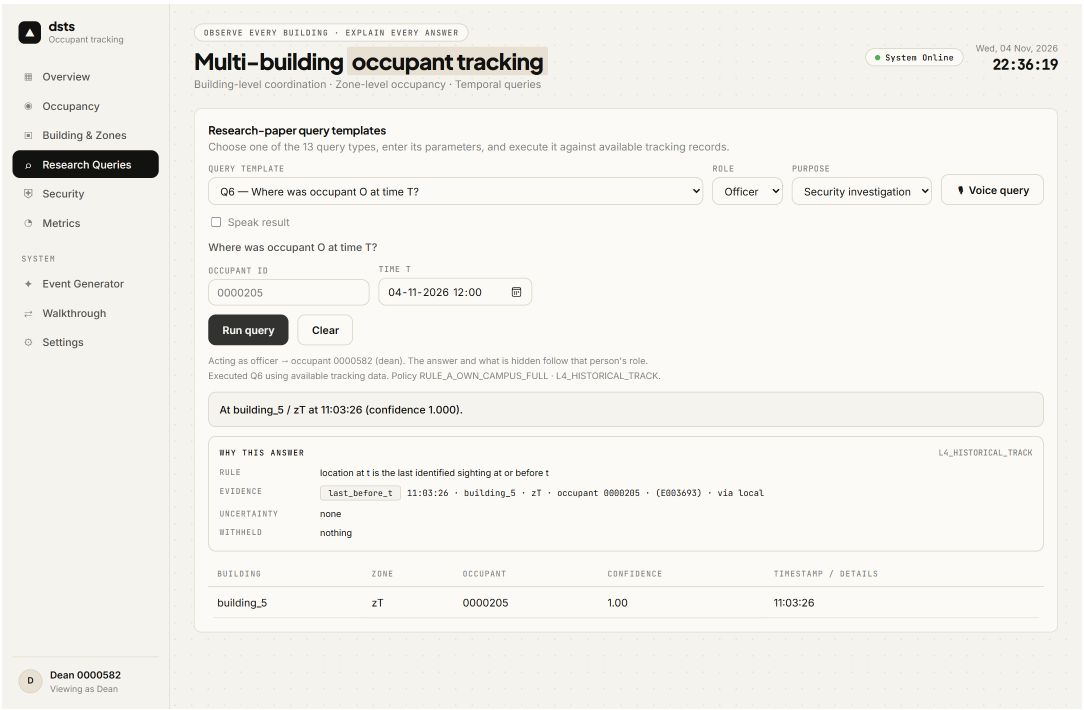


Figure 6.9: A Dean's query with its "Why this answer" card (rule, evidence, uncertainty, withheld)

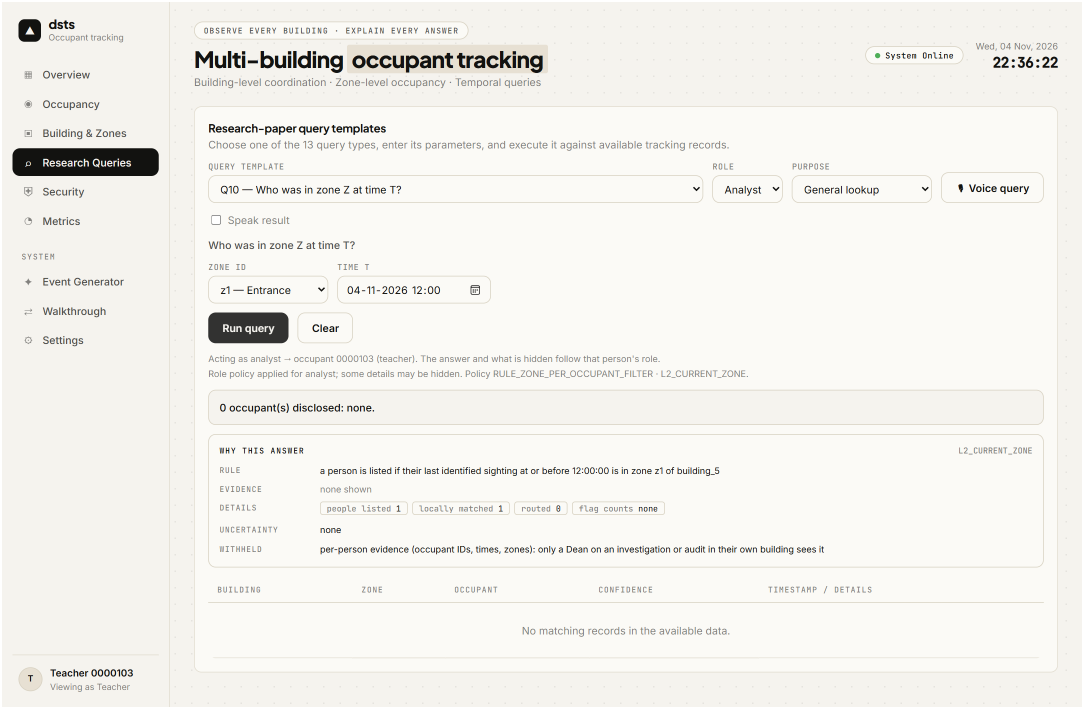


Figure 6.10: The same page for a Teacher: the answer is reduced and the card states what was withheld

Table 6.17: Comparison with the baseline and the approaches in the literature

Approach	**Visitor lookup**	**Face data leaving a building**	**Result here**
Centralised search over all galleries (baseline)	One global search	Galleries pooled in one store	92.24%; 100% of the gallery searched
Decentralised LSH + Bloom routing (this work)	Rank by Bloom score, verify top 5	Only a 7.3 KB Bloom filter; references move only after a confirmed match	91.54%; 10.3% of the gallery
DSTS base paper [6]	Central master list	Master list of visitor embeddings	Not re-run; replaced by the row above
Edge-camera tracking [1, 2, 3, 4, 5]	Within one site	Not applicable	No cross-building identity, RBAC or explanations

correct identification for the decentralised policy that verifies all five shortlisted buildings against 95.50% for the centralised search. Those numbers and the ones in this chapter measure different things and should not be compared directly. The earlier study counts *routed visitor events only* (visitors that no occupant of the capturing building accepts), uses 10 buildings and 20 seeds, and compares against a central search over the 450 identities outside the capturing building. This report counts *all* 11,000 events (local, visitor and masked), uses 11 buildings including the masked building, counts the visitors served from the visitor database as identified, and averages five seeds. The accuracy of the decentralised pipeline on visitors alone (89.6%) is lower than 92.81% mainly because the masked visitors are included and because visitors accepted as locals are counted as errors, and the central search (92.2%) is lower than 95.50% because it has to separate 550 identities, including the masked ones, instead of 450.

6.5 Discussion and Interpretation

The results meet the objectives that can be measured. Decentralised identification is within 0.7 points of a central search overall while touching about 10% of the galleries, so the privacy and locality gain costs little accuracy (objectives 1 and 4). Most visitor events are cheap because repeat sightings the same day come from the day cache, which is why the number of buildings contacted per visitor event is well below one. Tracking is reliable at building level and good at zone level (objective 1).

The access model and the explanations are verified by tests, flag scores and a persona matrix rather than by accuracy figures (objectives 2, 3 and 5): every policy cell and every query class is exercised, explanations are checked to be faithful to their answers and to reveal no more than the answer does, and the flags catch about three quarters of the wrong identifications and four in five wrong answers for about eight per cent of extra latency (objective 3). The weak spot of the explanation layer is the visitor database, whose errors are rarely flagged.

Two findings change how the system should be described. First, the Bloom filters' real false-positive rate is about 3.5%, not the 1% configured, because LSH codes collide; routing remains useful because verification is exact, but the privacy argument must not rely on the small configured rate. Second, masked identification is limited by the face model and not by the routing design, and the mask handler that was meant to help does not do so: it never fires for masked people and slightly harms unmasked ones.

6.6 Limitations

- The network, cameras and the face detector are simulated. Arrays are plaintext in memory before sealing and after opening, and the certificate authority is a stand-in whose signature is a message authentication code, not a real public-key infrastructure.

- The API has no TLS unless a reverse proxy adds it, no browser security headers, and no encryption of the stored embeddings and state databases. No outside penetration test or fuzzing was done; the attack tests are those the team wrote.
- Token secrets, caches and lockout counters are per process, so they would not hold across several servers; the dashboard's demo mode lets any viewer act as any role by design.
- The masked building uses web-crawled photos with some wrong labels, and 20% of the masked probes are augmented copies. Masked results are not an accuracy estimate for new people.
- A wrong identity stored in the visitor cache stays for the rest of that day, is rarely flagged, and a visitor whose departure is not identified stays "present" until the day ends.
- The effective Bloom false-positive rate (about 3.5%) is higher than configured, and no formal privacy guarantee is claimed.
- Every visitor is a person registered in some building; people who are registered nowhere cannot be identified and are left unidentified.
- Only 550 people in 11 buildings were measured; larger scale is untested. The same 20 test photos per person are reused across days in multi-day runs.
- Results come from simulated movement on replayed photos, not from live video.

Chapter 7

Conclusion

7.1 Conclusion

This project built and evaluated a system that identifies and tracks occupants across buildings without pooling face data, answers queries according to the caller's role and relationship, and explains each answer. A visitor is found by routing a capture with LSH codes and per-building Bloom filters and verifying it only in the shortlisted buildings. On 11,000 events per day, averaged over five random populations, it identifies 91.5% correctly against 92.2% for a centralised search, touching 10.3% of the galleries; the 95% confidence interval of the accuracy is ± 0.20 points, and building-level state F1 is 0.950. A dynamic visitor database makes about two thirds of the visitor events free of routing. A two-layer class-teacher policy, sealed transfers and redacted explanations are implemented and covered by 571 automated tests. The uncertainty flags attached to answers catch about three quarters of the wrong identifications and four in five wrong answers for about 8% of extra latency, although they rarely flag errors that come from the visitor database. Masked people are the hard case at 59.1%, and the evidence points to the face model, not the routing, as the limit. Measuring the Bloom filters showed that their effective false-positive rate is about 3.5%, not the 1% configured, so the privacy argument rests on the one-way encoding and on exact verification. All six objectives are achieved in simulation (Table 7.1); the security work is a tested design on a simulated network and not a hardened deployment.

7.2 Future work

Two directions follow directly from the results.

Better masked recognition. Masked people are identified only 59% of the time. The next steps are a face model trained with masks instead of one trained on unmasked faces; a real mask detector, because the existing angle-spread heuristic flags the wrong people; enrolling a few real masked photos per person, which gave about five points of rank-1 accuracy in the

Table 7.1: Objective-wise status

Objective	**Planned**	**Status**	**Evidence**
1. Multi-building DSTS extension	Mid review	Achieved	BSTS over 11 buildings; building-level state F1 0.950
2. RBAC query interface	24/9/26	Achieved	Two layers, class-teacher model, every policy cell tested, 64-request persona matrix
3. Explainable responses	26/9/26	Achieved	Faithfulness cases, flag recall 74% on identifications, 8% overhead
4. LSH + Bloom routing	Mid review	Achieved	10.3% of gallery touched, 0.7-point accuracy cost; effective false-positive rate 3.5%
5. Security layer	2/10/26	Achieved (simulated network)	Sealed transfers, tokens, lockout, audit log
6. Evaluation	Final review	Achieved	Event and state TP/FP/FN/TN, five seeds, centralised baseline, explanation and Bloom measurements

offline tests; and using tracking context, such as who is already inside the building, to reduce masked errors. Each can be validated on the same seeds and the same masked building, with the masked F1 of Table 6.7 as the target and the unmasked F1 as a check that nothing else gets worse.

Unknown visitor re-identification. Today a visitor must be registered in some building, and a face that nobody knows is left unidentified. A future version should re-identify unknown visitors across buildings and days: keep one-way codes of the unknown faces (LSH with Bloom filters) under a temporary identifier, link their sightings into a track, and let an administrator later name or purge them. This needs retention and privacy rules, and a check that two different strangers are not merged, which can be measured with the same precision and recall used here once unregistered people are added to the simulated population.

References

[1] C. Hegde *et al.*, "Indoor group identification and localization using privacy-preserving edge computing distributed camera network," *IEEE J. Indoor Seamless Position. Navig.*, vol. 2, pp. 51–60, 2024, doi: 10.1109/JISPIN.2024.3354248.

[2] H. Kwon *et al.*, "A feasibility study on indoor localization and multiperson tracking using sparsely distributed camera network with edge computing," *IEEE J. Indoor Seamless Position. Navig.*, 2023, doi: 10.1109/JISPIN.2023.3337189.

[3] S. Yang, F. Ding, P. Li, and S. Hu, "Distributed multi-camera multi-target association for real-time tracking," *Sci. Rep.*, vol. 12, Art. no. 11052, 2022, doi: 10.1038/s41598-022-15000-4.

[4] S. Casao, A. Naya, A. C. Murillo, and E. Montijano, "Distributed multi-target tracking in camera networks," arXiv:2010.13701, 2021.

[5] B. Gaikwad and A. Karmakar, "Smart surveillance system for real-time multi-person multi-camera tracking at the edge," *J. Real-Time Image Process.*, 2021, doi: 10.1007/s11554-020-01066-8.

[6] L. Mohan and V. Menon, "Modelling large scale camera networks for identification and tracking: an abstract framework," *IET Comput. Vis.*, vol. 14, no. 7, pp. 426–433, 2020.

[7] V. Menon, B. Jayaraman, and V. Govindaraju, "The three Rs of cyberphysical spaces," *Computer*, vol. 44, no. 9, pp. 73–79, 2011.

[8] S. P. Nair, K. A. Reddy, P. K. Alluri, and S. Lalitha, "Face recognition and tracking for security surveillance," *J. Intell. Fuzzy Syst.*, vol. 41, no. 5, pp. 5337–5345, 2021, doi: 10.3233/JIFS-189856.

[9] Z. Wu *et al.*, "Learning-based tracking-before-detect for unconstrained indoor human tracking using RF signal," *IEEE Trans. Mobile Comput.*, vol. 24, no. 6, pp. 4777–4793, 2025, doi: 10.1109/TMC.2025.3529501.

[10] C. Rathgeb, F. Breitinger, C. Busch, and H. Baier, "On application of Bloom filters to iris biometrics," *IET Biometrics*, vol. 3, no. 4, pp. 207–218, 2014.

[11] F. Marzani, T. van Ede, G. Heijenk, and M. van Steen, "Head count: privacy-preserving

face-based crowd monitoring," arXiv:2604.14250, 2026.

[12] B. H. Bloom, "Space/time trade-offs in hash coding with allowable errors," *Commun. ACM*, vol. 13, no. 7, pp. 422–426, 1970.

[13] L. Fan, P. Cao, J. Almeida, and A. Z. Broder, "Summary cache: a scalable wide-area web cache sharing protocol," *IEEE/ACM Trans. Netw.*, vol. 8, no. 3, pp. 281–293, 2000.

[14] M. S. Charikar, "Similarity estimation techniques from rounding algorithms," in *Proc. 34th Annu. ACM Symp. Theory Comput. (STOC)*, Montréal, Canada, 2002, pp. 380–388.

[15] Q. Lv, W. Josephson, Z. Wang, M. Charikar, and K. Li, "Multi-probe LSH: efficient indexing for high-dimensional similarity search," in *Proc. 33rd Int. Conf. Very Large Data Bases (VLDB)*, Vienna, Austria, 2007, pp. 950–961.

[16] A. Kirsch and M. Mitzenmacher, "Less hashing, same performance: building a better Bloom filter," in *Algorithms – ESA 2006*, ser. LNCS, vol. 4168, Zürich, Switzerland, 2006, pp. 456–467.

[17] J. Deng, J. Guo, N. Xue, and S. Zafeiriou, "ArcFace: additive angular margin loss for deep face recognition," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Long Beach, CA, USA, 2019, pp. 4690–4699.

[18] J. Guo, J. Deng, A. Lattas, and S. Zafeiriou, "Sample and computation redistribution for efficient face detection," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, virtual event, 2022.

[19] D. Yi, Z. Lei, S. Liao, and S. Z. Li, "Learning face representation from scratch," arXiv:1411.7923, 2014.

[20] Z. Wang *et al.*, "Masked face recognition dataset and application," arXiv:2003.09093, 2020.

[21] R. S. Sandhu, E. J. Coyne, H. L. Feinstein, and C. E. Youman, "Role-based access control models," *IEEE Computer*, vol. 29, no. 2, pp. 38–47, 1996.

[22] H. Krawczyk and P. Eronen, "HMAC-based extract-and-expand key derivation function (HKDF)," RFC 5869, IETF, 2010.

[23] A. Langley, M. Hamburg, and S. Turner, "Elliptic curves for security," RFC 7748, IETF, 2016.

[24] S. Josefsson and I. Liusvaara, "Edwards-curve digital signature algorithm (EdDSA)," RFC 8032, IETF, 2017.

[25] M. Dworkin, "Recommendation for block cipher modes of operation: Galois/Counter Mode (GCM) and GMAC," NIST Special Publication 800-38D, 2007.

[26] M. S. S. Suresh and V. Menon, “A generic and scalable approach to maximize coverage in diverse indoor and outdoor multicamera surveillance scenarios,” *IEEE Trans. Syst., Man, Cybern., Syst.*, vol. 53, no. 2, pp. 1172–1182, 2022.

[27] United Nations General Assembly, “Transforming our world: the 2030 Agenda for Sustainable Development,” A/RES/70/1, 2015.

[28] Y. V. Madhilakath Madathil, D. A. Kiran, A. Rajesh, and L. Mohan, “Adaptive candidate routing for distributed occupant recognition across multiple building IoT-enabled smart spaces,” manuscript in preparation, Amrita School of Computing, 2026.

Appendix A

Source Code and Reproducibility

The complete source code, configuration and test suite of the project are in one Git repository: `github.com/yadhu-vipin/Occupant_Tracking_v2`, branch `test_7`, commit `a7bde2e4`. No passwords, tokens or personal data are stored in the repository; the datasets and model files are public and are downloaded separately (see below), and the login master secret is supplied through an environment variable.

A.1 Setup and execution

Dependencies. Python 3.12; the packages in `requirements.txt` (NumPy, pandas, cryptography, FastAPI, uvicorn, pydantic, prometheus_client, PyYAML, httpx, pytest). The face models are needed only to build the masked building and are not needed to run the pipeline on the stored embeddings.

Steps.

1. Clone the repository, create a virtual environment and run `pip install -r requirements.txt`.
2. Place the embedding corpus files `emb_arcface.npy` (20,000 $\times$ 512) and `meta.csv` in `corpus/`. They are not in Git because of their size.
3. Optional, for the 11th (masked) building: download the RMFRD archive [20] into `external/rmfd_download/` and the InsightFace buffalo_l model files (`det_10g.onnx`, `w600k_r50.onnx`) into `external/arcface_model/`, then run `python manage.py masked`. Without it, set `masked_building.enabled: false` in `config/experiment.yaml` to run the original ten buildings.
4. Run the experiment (Table A.1); every seed writes into its own folder `runs/seed_XX/`.

Configuration. All settings of an experiment are in `config/experiment.yaml`: the population (10 buildings of 50, default seed 42), the masked building switch, the event model

Table A.1: Main commands

Command	**What it does**
python manage.py test	Runs the 571 automated tests
python manage.py masked	Builds the masked building (corpus/masked) from RMFRD
python manage.py pipeline –seed 42 –days 1	One population: enrol, events, recognition, routing, state, evaluation
python manage.py run –seeds 0-4 –days 1	Five populations and the cross-seed summary (runs/summary.json)
python -m centralized.compare_all	Centralised search against the pipeline over every event of a run
python -m evaluation.evaluate_explanations	Explanation flags against ground truth and explanation overhead
python tools/measure_bloom_fpr.py	Effective false-positive rate of the Bloom filters
python tools/rbac_matrix.py	Persona matrix: 64 requests with expected decisions
DSTS_DEMO_MODE=1 python manage.py serve –seed 42	Starts the API and the dashboard on port 8000

(default 14 simulated days; the results in this report use one day, `-days 1`, with 8 events per occupant and a visitor share of 20%) and the visitor promotion rule. The federation contract (hash functions, Bloom size, thresholds) is in `shared/params.json` and `shared/routing_params.json`; every artifact carries a hash of it, and a mismatch is rejected.

Time and outputs. A one-day run of 11,000 events takes about eight minutes on a laptop CPU. Table A.3 lists what each run writes.

A.2 Supporting material

A.2.1 Repository layout

A.2.2 Files written by one run

A.2.3 Where each number in this report comes from

A.2.4 Visitor outcomes in the detailed run

Table A.5 gives the counts behind Figure 6.2 (seed 42, 2,200 visitor events).

A.2.5 Dataset documentation

CASIA-WebFace subset. 500 identities with 40 embeddings each, split into 20 reference and 20 test embeddings; embeddings were extracted once with the InsightFace buffalo_l

Table A.2: Repository layout

Folder	**Content**
population/	Seeded population, roles, rosters, corpus assembly
buildinglib/	Enrolment, Bloom filter, routing, verification, visitor pool
pipeline/	Event generation, recognition, routing and recognition, state, query engine, explanations
recognition/	Local recognition contract and the mask handler
dsts/	State library, FastAPI server, dashboard
access_control/	Tokens, credentials, lockout, tickets, two-layer policy, audit log
secure_transfer/	Sealed payloads, certificates, replay guard, handler
evaluation/	Event, state and explanation metrics; cross-seed summary
centralized/	Centralised baseline and the all-event comparison
tools/	Masked-building builder, Bloom measurement, persona matrix
tests/	The automated tests, grouped by area
runs/seed_XX/	Generated per seed; not in Git

Table A.3: Outputs of one run (runs/seed_XX/)

File	**Content**
population.json, corpus_meta.csv, roles/	Who is in which building; roles and class rosters
filters/building_N.npz	The published Bloom filters and their manifests
events/generated_events.json, ground_truth.json	Observable events and their true labels
events/state_ground_truth.json	The true campus state after every event
recognition/, retrieval/	Local recognition results and routing attempts
state/phase4_results.json	Identified events with their provenance
nodes/building_N/	Per-building databases: registered, visitor, pointers, permanent visitors
results/event_metrics.json, state_metrics.json	Event and state confusion metrics
results/end_to_end_summary.json	Accuracy, routing recall, visitor outcome buckets, masked cohort
results/centralized_vs_decentralized.json	Central comparison per cohort
results/explanation_flags.json, explanation_overhead.json	Explanation measurements
results/bloom_fpr.json	Bloom false-positive measurement

Table A.4: Source of the reported results

Result	**Produced by**
Tables 6.2, 6.3, 6.4, 6.5, 6.7	evaluation.evaluate_events and centralized.compare_all, averaged by evaluation.summarize_runs (five seeds)
Table 6.6	evaluation.evaluate_states against events/state_ground_truth.json
Tables 6.10–6.13	evaluation.evaluate_explanations
Table 6.14	tools/measure_bloom_fpr.py
Tables 6.8, 6.9	Offline masked-probe experiment on building 11 (one-off script; numbers recorded in this report)
Table 6.15 and Table 6.16	pytest and tools/rbac_matrix.py

Table A.5: Visitor outcome buckets, seed 42

Outcome	**Events**
Correct, served from the visitor database	1,431
Correct, routed	540
Right building reached, still wrong	56
Routing miss, abstained	90
Visitor accepted as a local occupant	45
Wrong identity, served from the visitor database	16
Routing miss, wrong identity	8
Verification failure at the true building	8
Ranking failure, wrong identity	6

models and are stored as 512-dimensional unit vectors. **RMFRD subset.** The 50 identities with the most real masked photos that also have at least 20 unmasked photos. Each has 20 distinct real unmasked photos as references and 20 masked probes: the real masked photos (10 to 20 per person, median 16; 15 people have 20) topped up with augmented copies of them. Photos are detected and aligned with SCRFD; the augmentation is seeded from the identity name, so the build is reproducible. The masked data are not in Git (a `.gitignore` rule covers `corpus/masked/` data and `external/`).